HOCHSCHULE
HANNOVER
UNIVERSITY OF
APPLIED SCIENCES
AND ARTS
–
Fakultät IV
Wirtschaft und
Informatik

Fortgeschrittene Aspekte verteilter Systeme

Domain Driven Design

Hans Grönniger



“There should be some learning when a domain model is discussed.”

- Eric Evans



Where do we stand in the ShareWise project?

So far

- Defined Cloud computing
- Got to know ShareWise as a case study
- Gone through the Mallorca user journey

Today, we learn how to

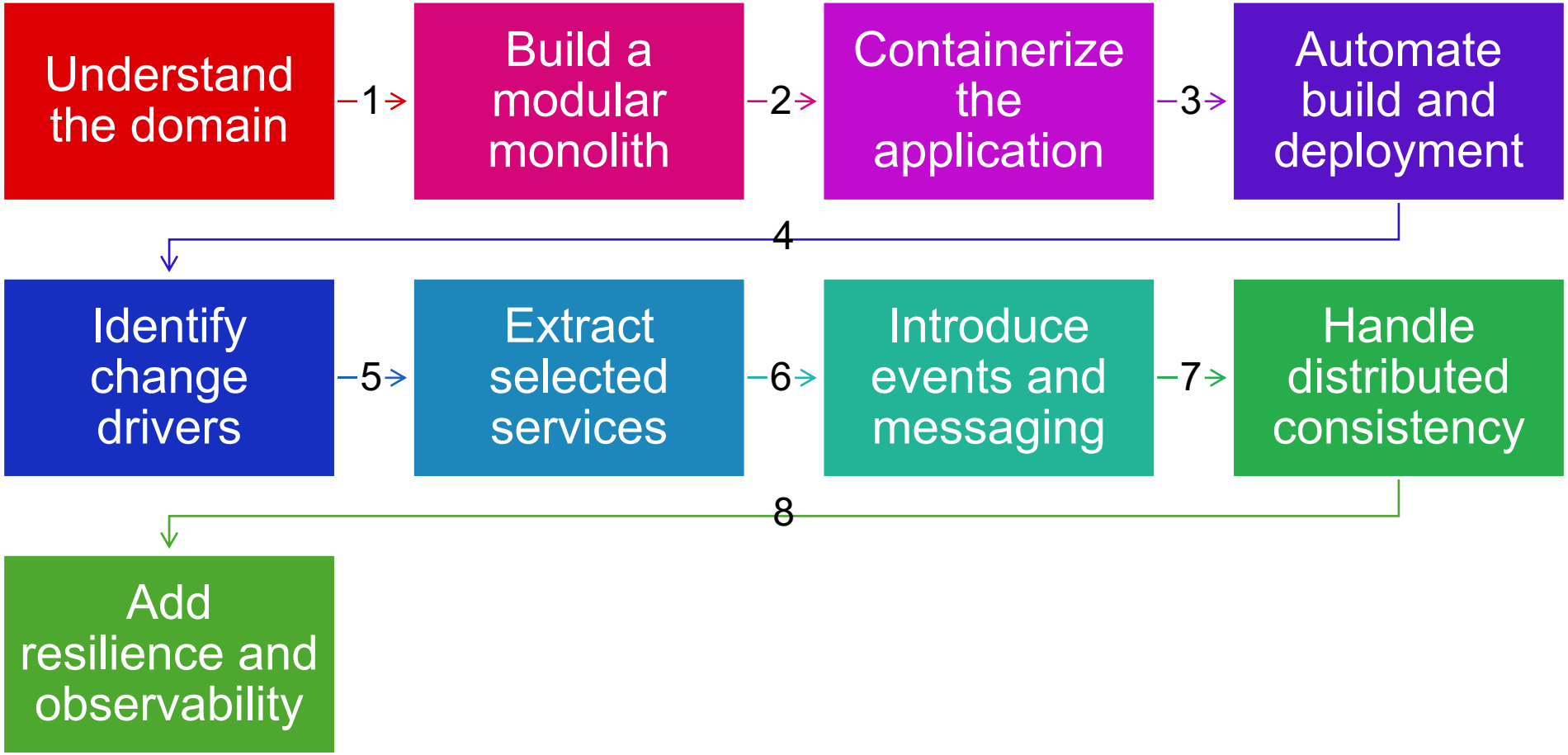
- Specify the language of the domain
- Make business processes visible
- Identify business boundaries
- Discover rules and invariants

Up Next

- Implement ShareWise as a modular monolith



ShareWise: From modular monolith to cloud-native system



User Journey



1. A user registers
2. The user creates the group “Mallorca 2027”
3. Friends are invited
4. The invited users join the group
5. Members record shared expenses
6. Expenses are corrected or removed when necessary
7. Members review their current balances
8. Members record settlement payments
9. An authorized member closes the group for settlement
10. The group is archived

User Journey



What are the business rules for this step?

1. A user registers
2. The user creates the group “Mallorca 2027”
3. Friends are invited
4. The invited users join the group
5. Members record shared expenses
6. Expenses are corrected or removed when necessary
7. Members review their current balances
8. Members record settlement payments
9. **An authorized member closes the group for settlement**
10. The group is archived

ShareWise User Journey

„An authorized member closes the group for settlement“

What are the rules for this step?

- Who is authorized?
- Can expenses be added afterwards?
- Can expenses still be corrected?
- Need all balances be 0 before closing?
- Does archiving happen automatically afterwards?
- Can closing be revoked?



Common Business Model

Same words different concepts, different understanding

Possible consequences

- contradictory requirements
- Unclear business rules
- incorrect data models
- Unclear responsibilities
- Special cases in the application code
- unstable module boundaries
- inappropriate service limits

Before we technically divide into modules, we need a common business model.
And agree on the meaning of words. Work on a common “mental model”.

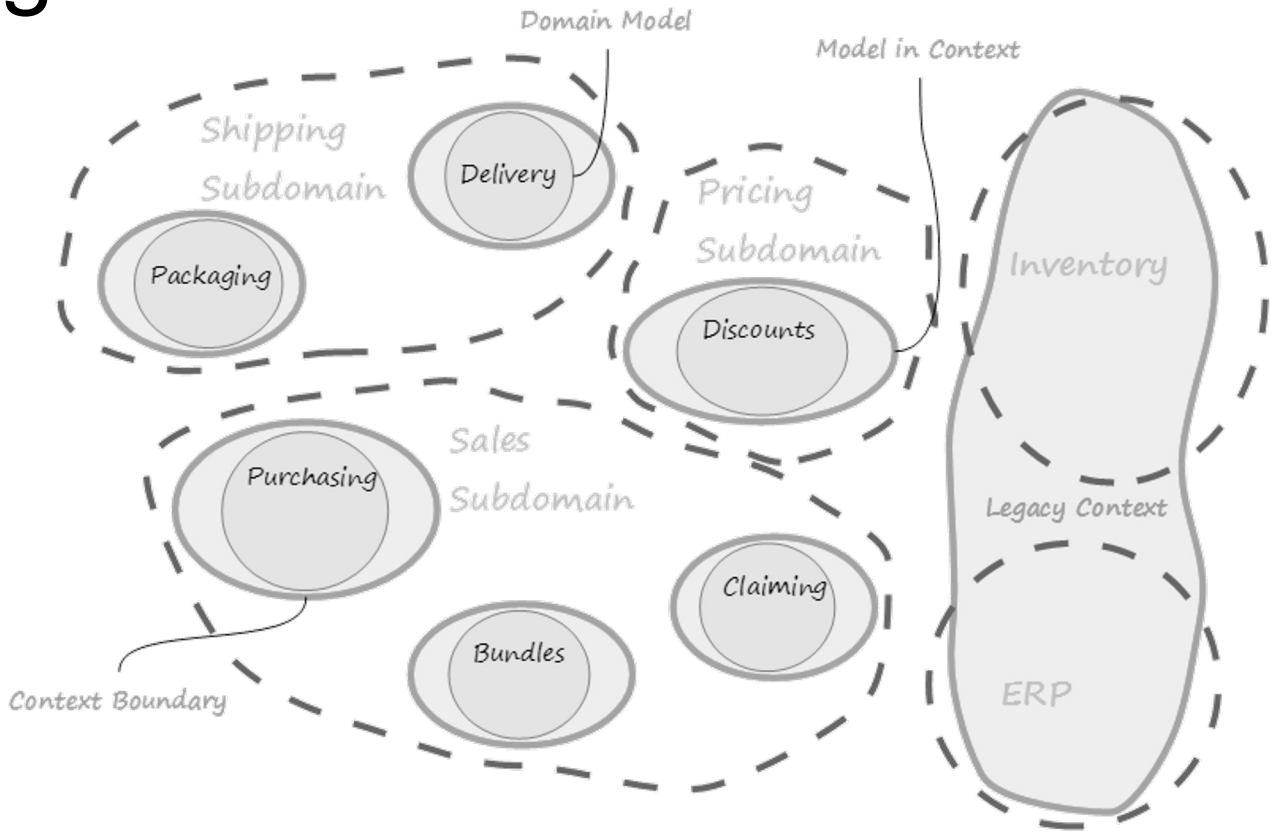


Domain Driven Design

Domain Driven Design
- What is it?
- Effective Software Design
- Strategic Design
- Tactical Design

Strategic Design
- Subdomains
- Bounded Context + Ubiquitous Language
- Context Mapping

Tactical Design
- Business Logic Pattern
- Architectural Pattern



Slides adapted from:
Nane Kratzke, Cloud-native Computing, 2024

Domain Driven Design

Domain Driven Design

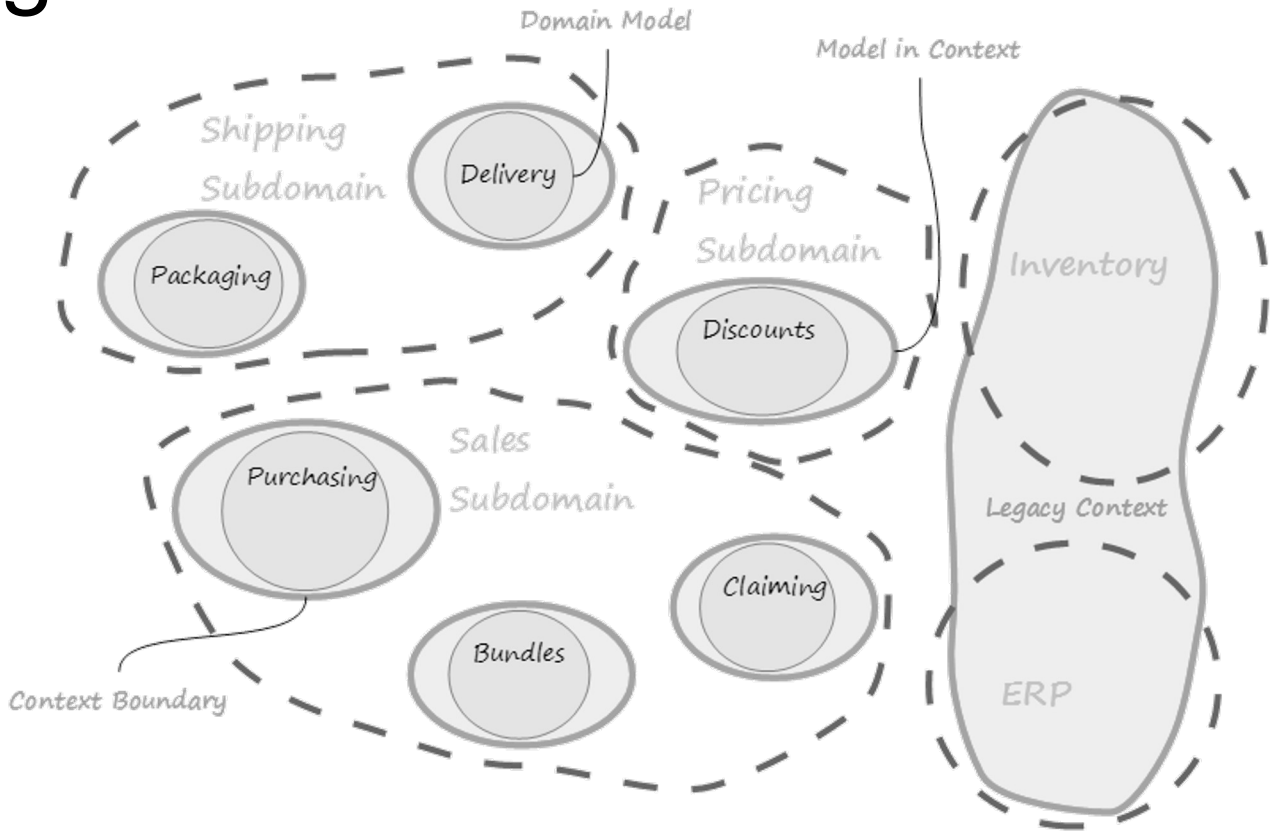
- What is it?
- Effective Software Design
- Strategic Design
- Tactical Design

Strategic Design

- Subdomains
- Bounded Context + Ubiquitous Language
- Context Mapping

Tactical Design

- Business Logic Pattern
- Architectural Pattern



Domain Driven Design

On the quest for a good design methodology for cloud-native systems

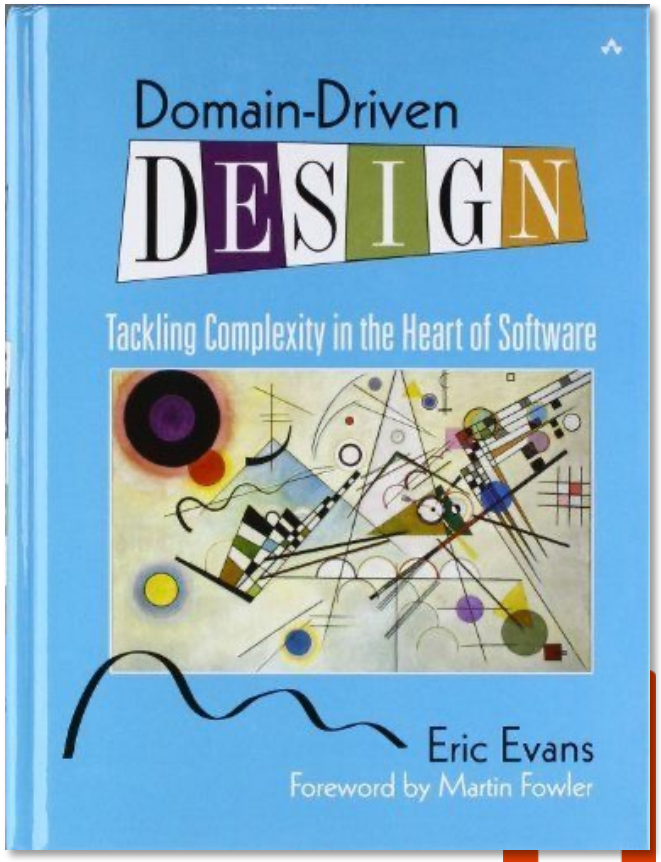
Well-know Principles

- Decomposition (into services)
- Loose coupling (Standard formats, e.g. JSON/XML, Messaging)
- High cohesion (in modules) (single responsibility principle)
- Independent deployability
- Conways Law
- You build it, you run it
- …



Domain Driven Design

- Domain-driven design (DDD) is a methodology for modeling complex software
- Major concern is to understand the business logic.
- For the design to be effective, it has to be built around a model of the application domain, the domain model.
- DDD is oriented towards agile development but does not require it
- However, iterative software development and close collaboration between developers and domain experts is required.



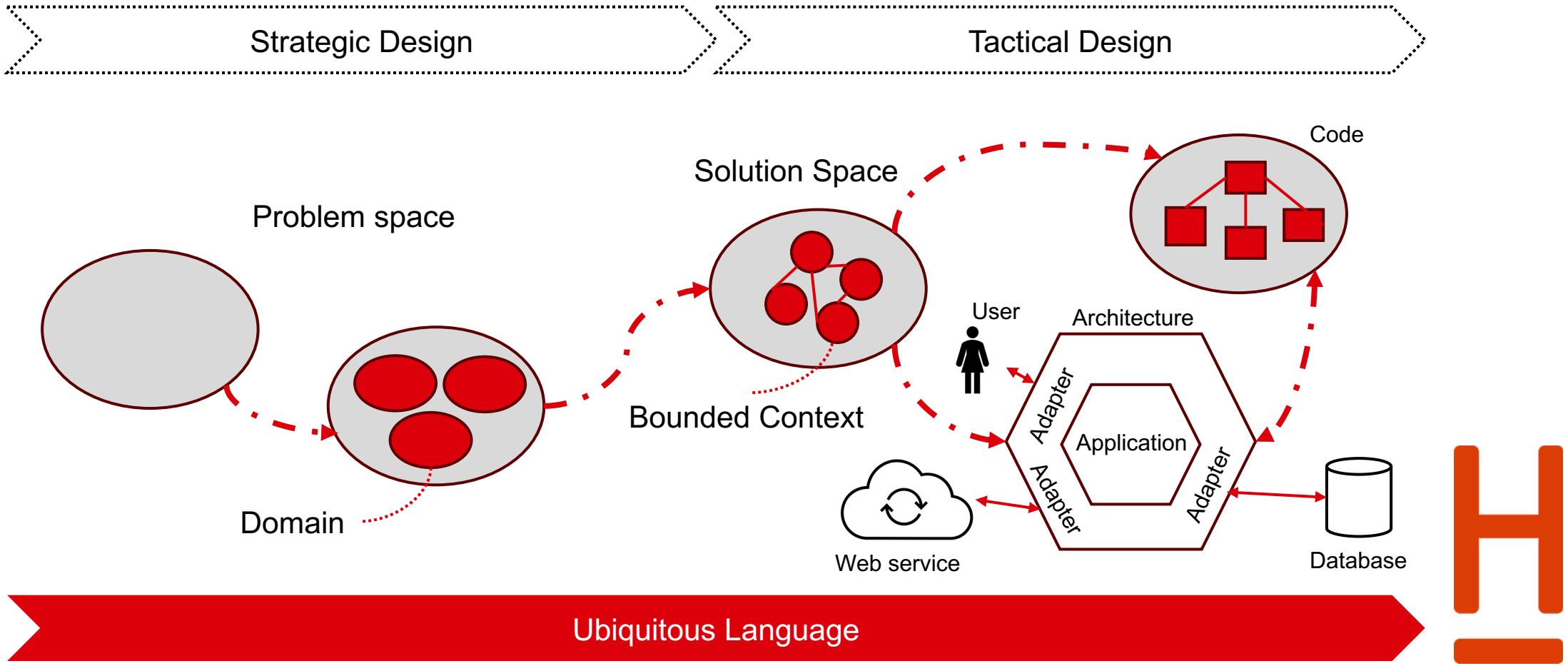
Domain Driven Design – Effective Design

- Strive not for a perfect but an effective design
- Fulfil requirements up to a point so that the software creates business value (e.g., be better than competitor in the core aspects)
- Push organizations to think what makes the software create said business value
- In that core domain of requirements, the efforts are focused
- Other areas should be coved with standard software or external services

"Most people make the mistake of thinking design is what it looks like. People think it's this veneer—that the designers are handed this box and told, "Make it look good!" That's not what we think design is. It's not just what it looks like and feels like. Design is how it works."



Domain Driven Design Overview



Domain Driven Design

How to find out what's important?

Strategic Design

- What is important to reach our business goals?
- How do we divide the work based on priorities?
- Where do we consolidate?

Approach

- Involve Domain Matter Experts using a common language (Ubiquitous Language)
- Divide complex domain models into subdomains and bounded contexts
- Clarify the integrations between subdomains using context mappings



From this, ultimately services are defined and teams are formed according to Conway's law

Domain Driven Design

How to break down what's important

Tactical Design

- "Tactical design is like using a thin brush to paint the fine details of your domain model." (Vaughn Vernon)
- Transition from problem to solution space
- Work out the details of a bounded context

Approach

- Working inside a bounded context
- Combining entities and value objects into aggregates
- Model and implement service interactions across bounded contexts using domain events



Domain Driven Design – Everywhere?

„There is a life beyond DDD. Not every good design needs to be Domain-Driven (though I can accept it should always be driven by the domain, just not necessarily in the DDD sense). ***You can design good systems even if you're not a DDD expert.“***

Stefan Tilkov
Blog Post



Stefan Tilkov
Managing Director and Principal Consultant at INNOQ; Strategic consulting in the field of software architectures

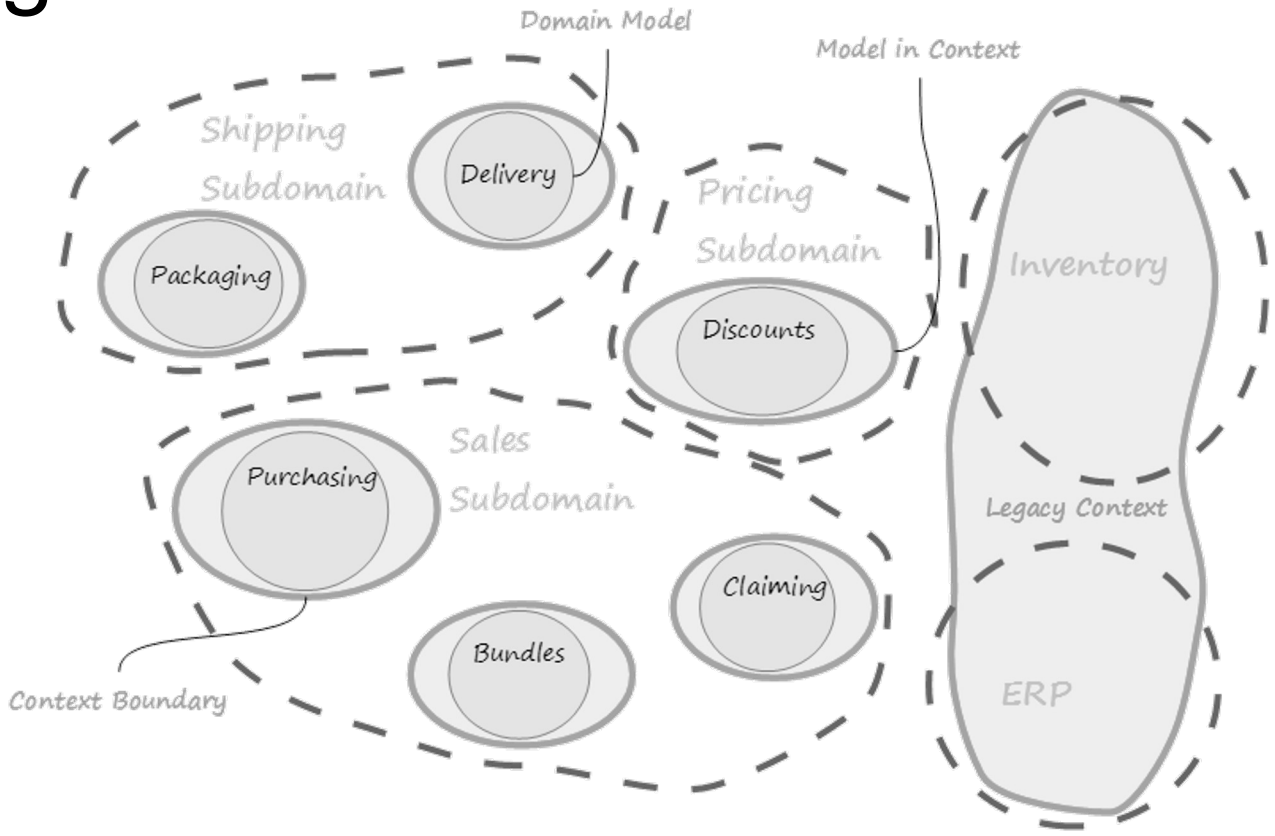
Siehe auch: Stefan Tilkov, **GOTO 2019:** Good Enough Architectures, https://www.youtube.com/watch?v=PzEox3szeRc

Domain Driven Design

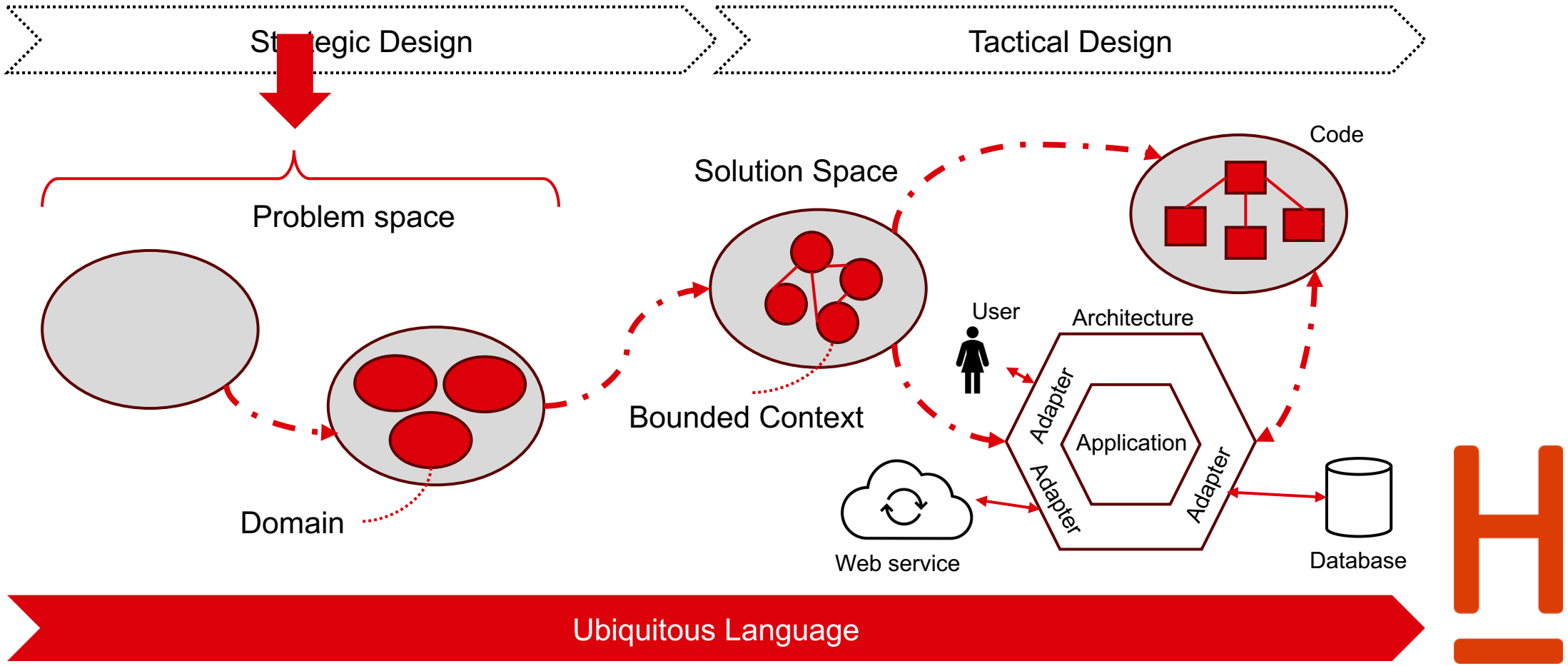
Domain Driven Design
- What is it?
- Effective Software Design
- Strategic Design
- Tactical Design

Strategic Design
- Subdomains
- Ubiquitous Language
- Bounded Context
- Context Mapping

Tactical Design
- Business Logic Pattern
- Architectural Pattern



Domain Driven Design



Strategic Design

Analysis of Business Domains

A business domain defines the area of activity of an organization or company. In general, it is the service is offered to customers.

A company can operate in multiple business domains. For example, Amazon offers both retail and cloud computing services.

Companies may also change their business areas over time. Nokia, for example, has been active in such diverse areas as woodworking, rubber manufacturing, telecommunications and mobile devices throughout its history.

Main business domain?









Strategic Design

Analysis of Business Domains

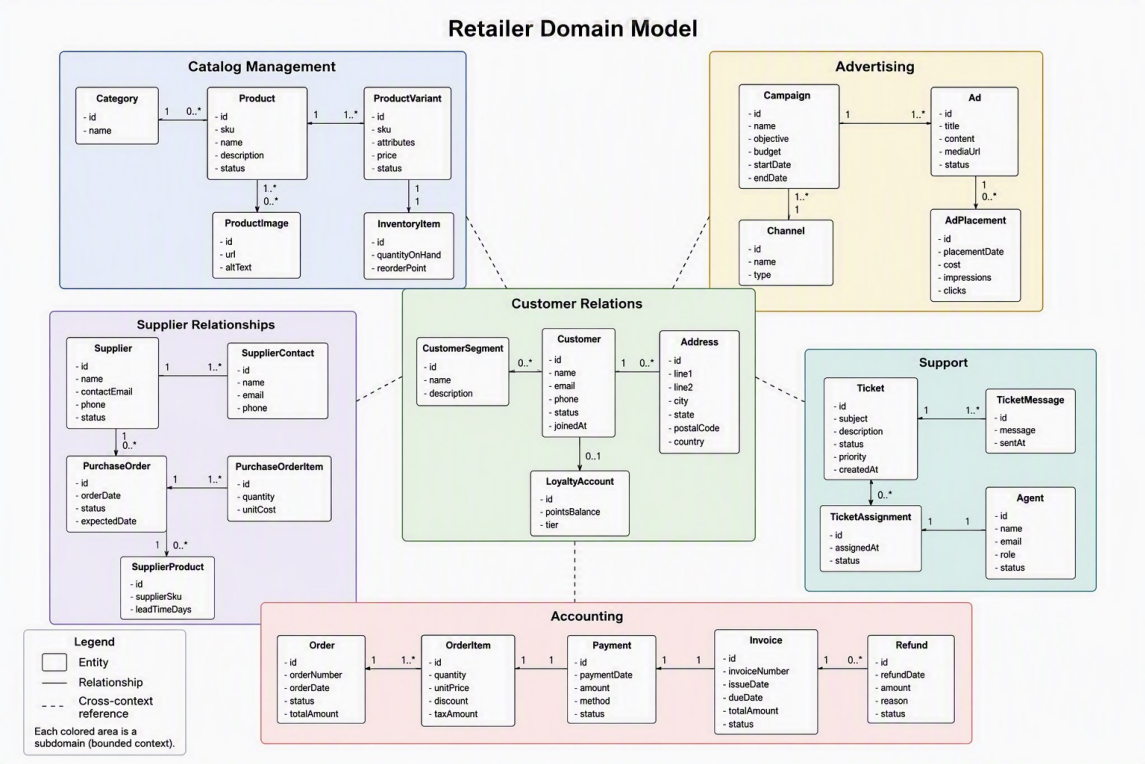
To achieve the goals of its business domain, a company must operate in multiple subdomains.

A subdomain is a fine-grained area of business activity.

All subdomains together make up the company's business domain.

Often, subdomains correlate with the company's departments or other organizational units.

For example, an online retail store might include subdomains such as catalog management, advertising, accounting, support, customer relations, supplier relationships, and others.



Strategic Design

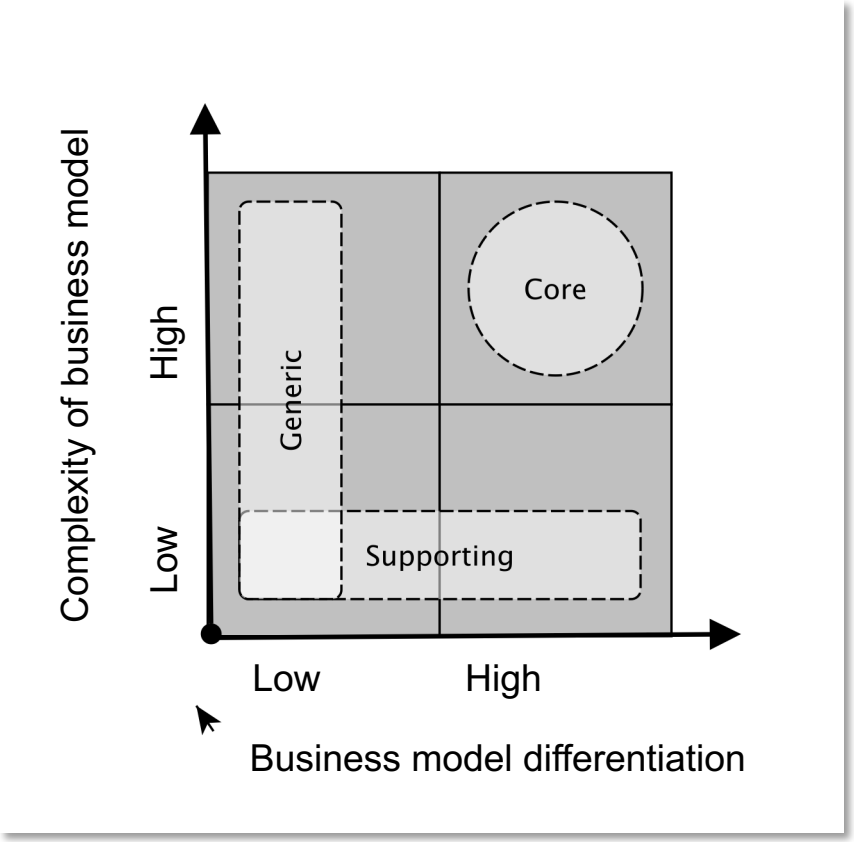
Different domain types

A domain is generally understood to be a specific area of business activity or knowledge.

The term is very broad and abstract.

To make it more concrete and tangible, it makes sense to divide it into smaller parts called subdomains.

Subdomains come in 3 types.



- Core Domain
- Supporting Subdomain
- Generic Subdomain

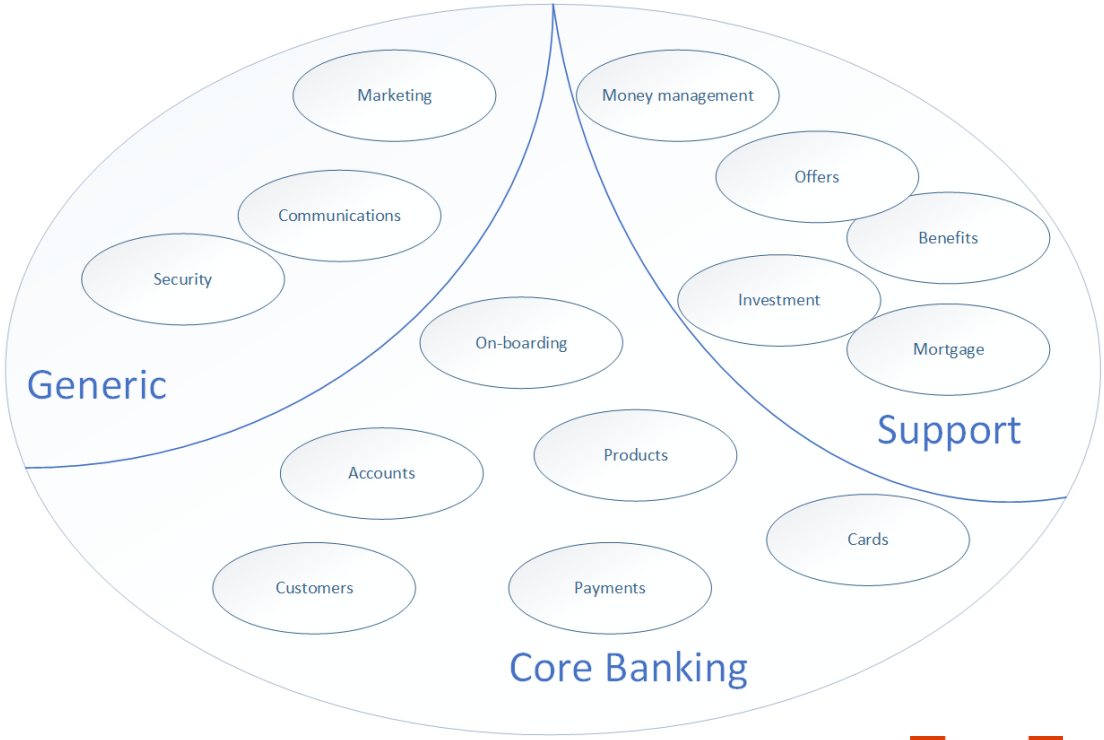
All subdomains, regardless of category, are important for overall solutions. If a domain is missing, the solution will be incomplete. However, the domains require different effort and can also have different requirements for quality and completeness.



Strategic Design

What is a Core Subdomain?

- Core domain differentiates companies from competitors through new products or process optimizations
- Easy-to-implement core subdomains are not sustainable, so they are usually complex with high barriers to entry
- Investments in system and software development should primarily go into the core domain
- Core subdomains must be implemented internally and cannot be purchased or adopted
- Outsourcing the implementation of a core subdomain isn't a good idea.



Strategic Design

What is a Supporting Subdomain?

Supporting subdomains do not provide a competitive advantage for businesses.

They merely support the company's core business.

- Do not have complex business logic.
- Often based on simple ETL or CRUD interfaces
- Do not change very often, are stable.

Example core domain of an online advertising company:

- Tailoring ads to visitors
- Ad effectiveness optimization
- Minimization of advertising costs

Supporting domain of an online advertising company:

- Managing creative materials (does not affect profits)

Implementation?

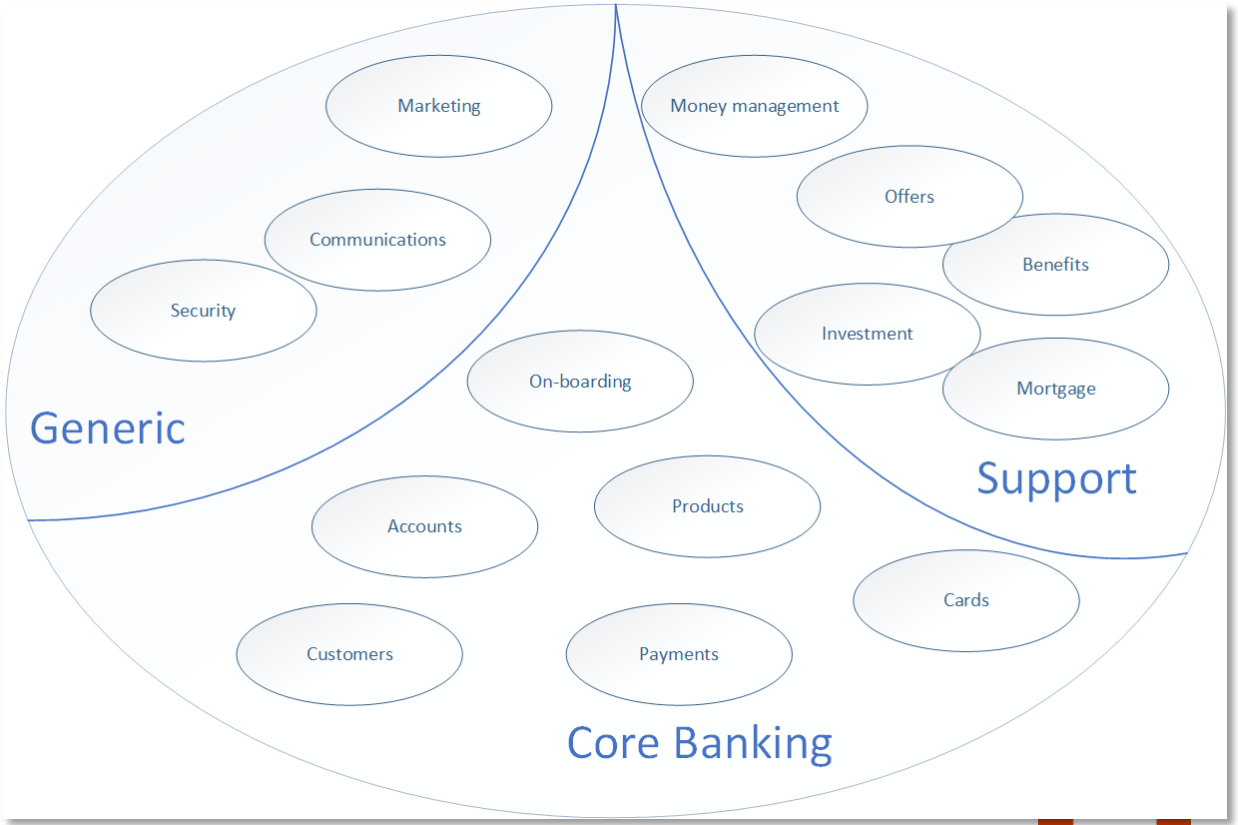
- Try to avoid to implement yourself due to lack of competitive advantage
- Often no ready-made solutions, then use comprehensive application development frameworks
- The simplicity of the business logic makes supporting subdomains an obvious candidate for outsourcing



Strategisches Design

What is a Generic Subdomain?

- Generic subdomains also show complexity
- They do not provide a competitive advantage to a company because there are already proven and widely used implementations („commodity“)
- Generic subdomains are business activities that all companies carry out in the same way
- Therefore, attempts are usually made to cover these with standard software and systems
- Examples would be identity management with Microsoft's Active Directory or Oracle's database products, or SAP's products such as R/3 or S/4 HANA



Strategic Design

Domain types - summary

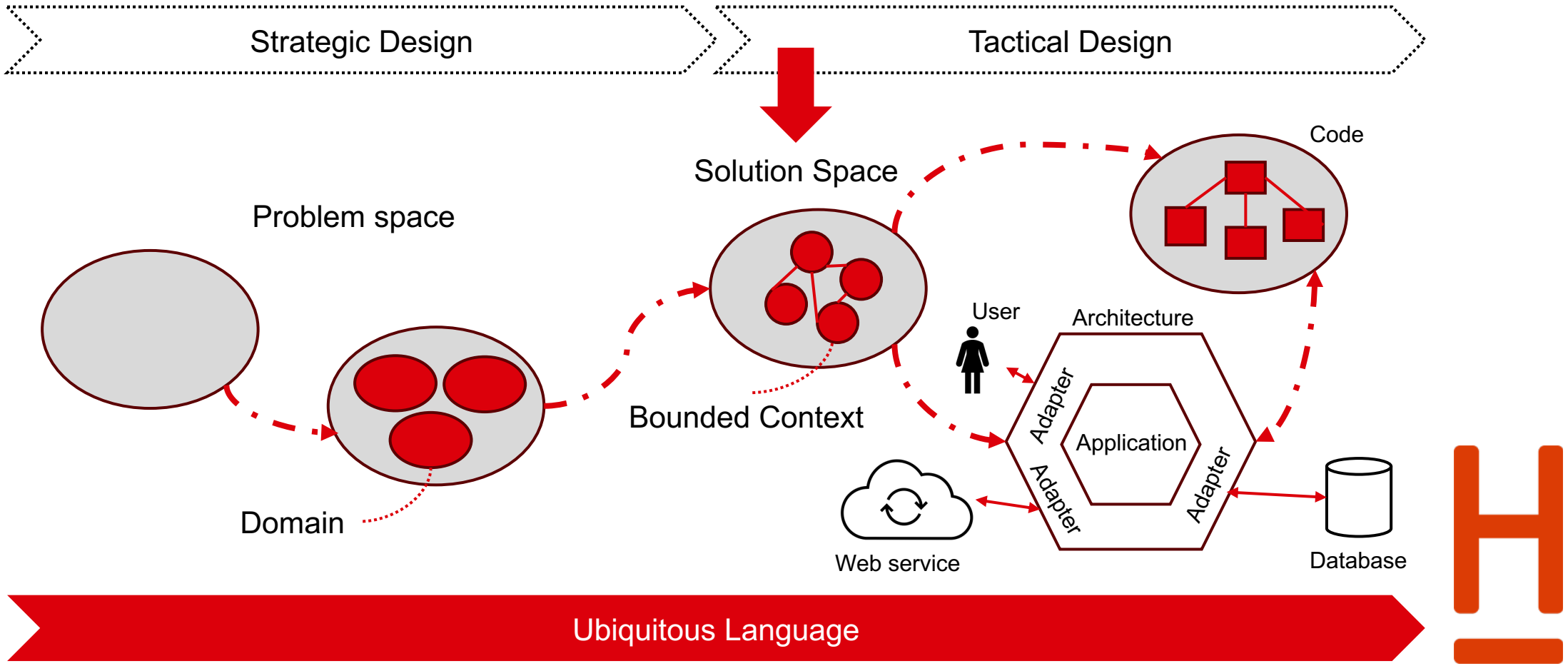
	Core Subdomain	Supporting Subdomain	Generic Subdomain
Unique Selling Point / Business Critical	Yes	No	No
Company-specific	Yes	Yes	No
Technical complexity (barrier to entry)	High	Low	High
Technical Rate of Change	High	Low	Low
Exist Solutions / Products / Services	No	Partly	Yes
In-house development makes sense	Yes	Yes	No
Outsourcing of development makes sense	No	Yes	No
Do COTS products / managed services exist?	No	Partly	Yes

ShareWise Domains?

- Discuss in groups.



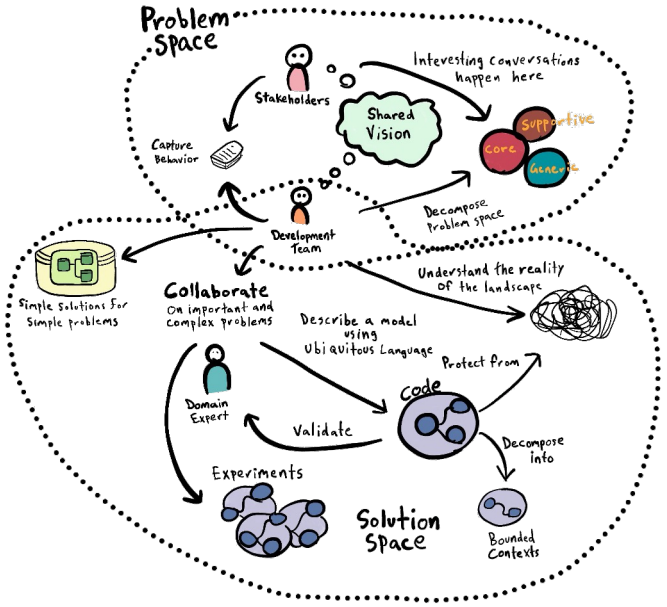
Domain Driven Design



From problem space to solution space

Ubiquitous Language

- A domain is sometimes referred to as a "problem space" because it defines business problems that software is supposed to solve.
- The problem space focuses on business problems
- The solution space focuses on technical solutions
- The subdomains belong in the problem space.
- The relationship between the problem space and the solution space is difficult to maintain.



Common software development methodologies are usually implicitly based on the following translations:

- *Domain Knowledge in Analysis Models*
- *Analysis models in requirements*
- *Requirements in System Design*
- *System design in source code*

Translation errors can occur at any of these transitions.

Codification of domain knowledge

Ubiquitous Language

- Software modeling often involves activities in which domain knowledge is translated into an "engineer-friendly" form
- Such artifacts (e.g. in UML class diagrams) are called analysis models and describe the requirements for the system
- Analytics models are less about understanding the business domain and more about structuring the solution
- This translation is dangerous, as information can be lost: Domain knowledge is lost on the way to software engineers

Since the software crisis in the 1960s, many studies have investigated the failure of software projects. Almost all of them have shown that communication, among other things, is essential for project success.
Almost all software development models therefore try to pass on domain knowledge from domain experts to engineers. This is usually done through intermediary roles that "pass on" and translate knowledge from the domain to the development department: system/business analysts, product owners, project managers, etc.
However, this does not automatically imply effective, i.e. understanding-based, communication.
Domain-driven design attempts to bring the knowledge of domain experts to software engineers by means of a ubiquitous – and jointly developed – language, the so-called Ubiquitous Language.

Strategic Design

Concepts in Problem Space - Ubiquitous Language



- Common language is key to common understanding
- Complex but necessary step
- Clarification of technical terms
- Clarity of common language is an indicator of the depth of understanding of the problem space.
- Unclear terms indicate deeper and hidden (possibly misunderstood) concepts.
- Implicit assumptions must be made explicit in order to be able to define a clear meaning within the ubiquitous language.

Ubiquitous

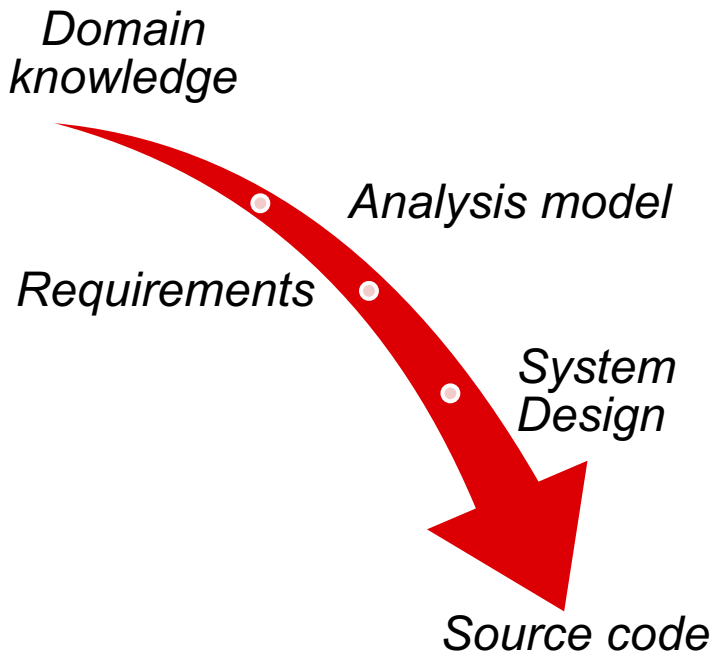
- Everyone in the project speaks and understands the same language
- All relevant concepts are described in the language
- The language is being developed evolutionarily and is present in all phases of development (also in the code!)

One of the most important concepts in DDD

- Basis for common understanding
- Basis for end-to-end understanding
- Indicator of domain penetration and descriptive stability

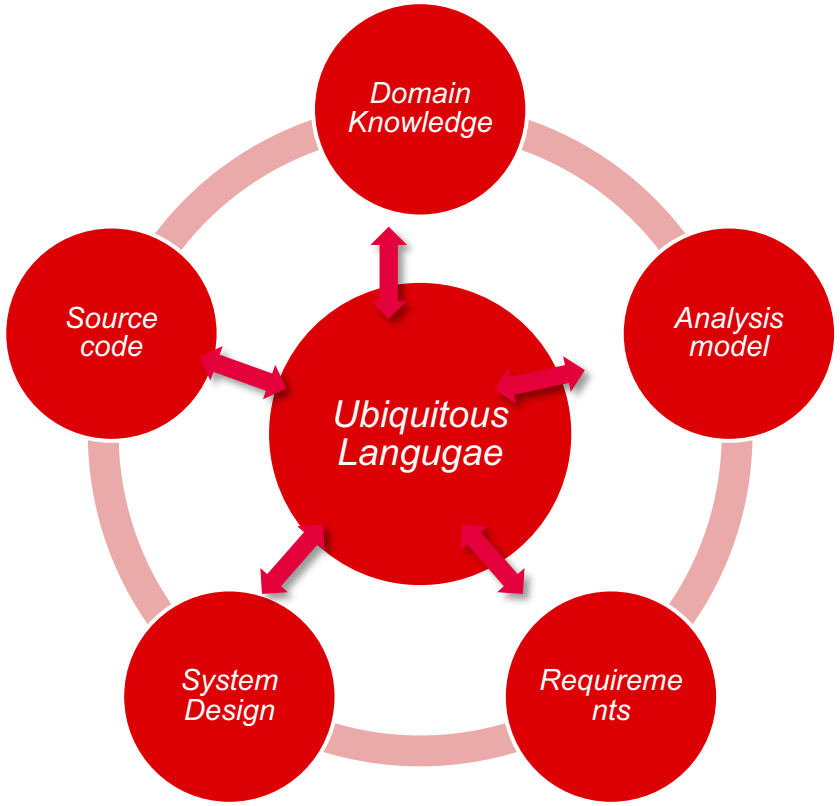
Ubiquitous Language

Non - DDD



Instead of constantly translating domain knowledge, DDD calls for cultivating a single language to describe the business domain: the ubiquitous language.

DDD



Through the consistent use of ubiquitous language, a common understanding can be cultivated between all project stakeholders. The ubiquitous language should only consist of terms that relate to the business domain.

Ubiquitous Language

Ambiguous terms

Policy can mean “legal regulation“ or „insurance contract“. What do we do in case of an insurance company? Policy is ambiguous.

Ubiquitous language requires us to define a single meaning for each term.

We could go with "regulatory provision" and "insurance contract" (but not policy).

Synonymous terms

Two terms cannot be used interchangeably.

For example, in many systems we may use the terms "user”, "visitor", "administrator", "account", etc. interchangeably.

Synonymous terms usually refer to different detailed concepts.

For example, the terms "visitor" and "account" technically may refer to unregistered and registered users of the system. Understanding the differences between the terms used allows for the creation of simpler and clearer models and implementations of the business domain entities.

Ubiquitous Language

Like so

The following statements are formulated in the language of marketing domain experts

- An advertising campaign can display various creative formats (videos, banners, animations).
- A campaign can only be published if at least one of its formats is active.
- Sales commissions are charged for approved transactions.

But not like

The following statements are purely technical

- The ad iFrame displays an HTML file that embeds content linked by a URL.
- A campaign can only be published if it has at least one associated record in the active-placements table.
- Sales commissions are based on correlated records from the transactions and approved sales tables.



Ubiquitous Language

Like so

```
closeGroupForSettlement()
recordSettlementPayment()
calculateFinalBalances()
archieveGroup()
```

But not like

```
updateStatus()
processData()
handlePayment()
calculate()
```



ShareWise Language?



- Expense
- Share
- Balance
- Debt
- Settlement

- Business rules (invariants)

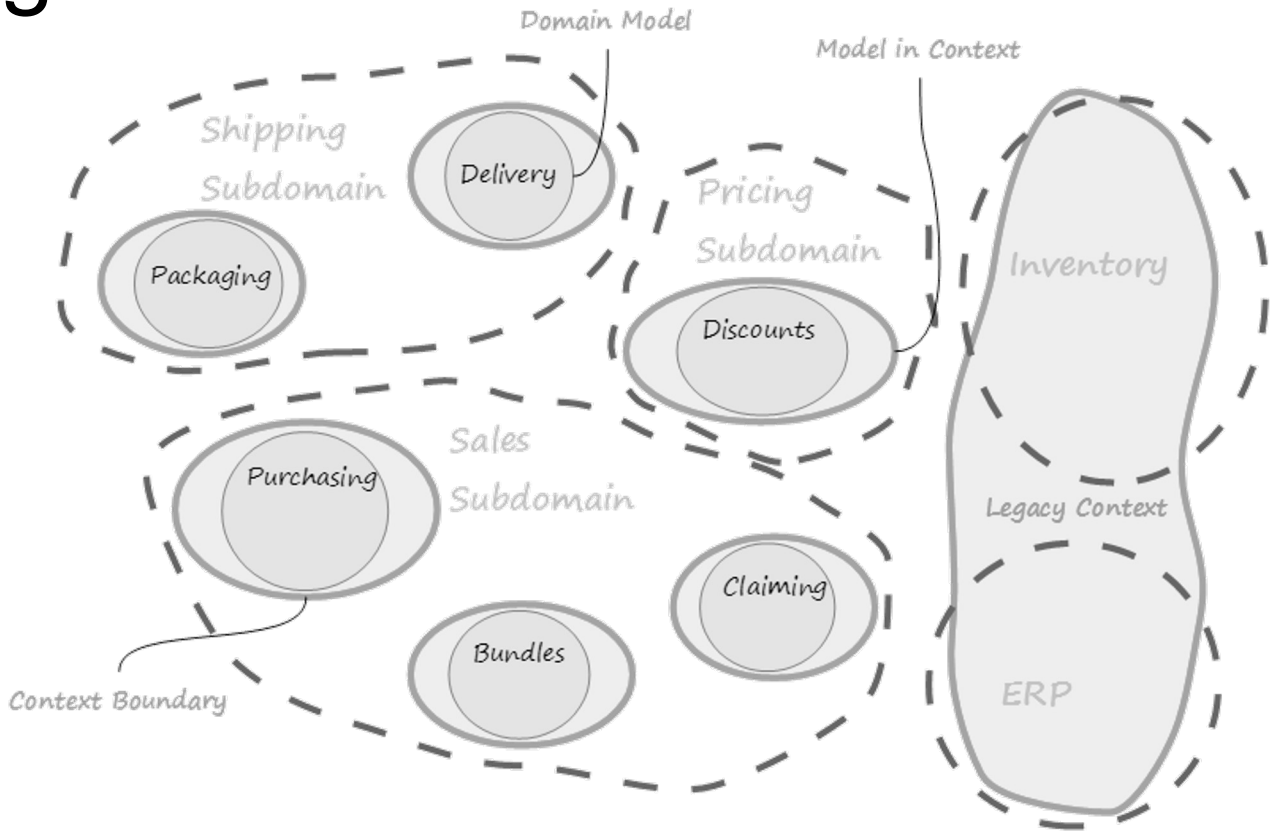
- Glossary

Domain Driven Design

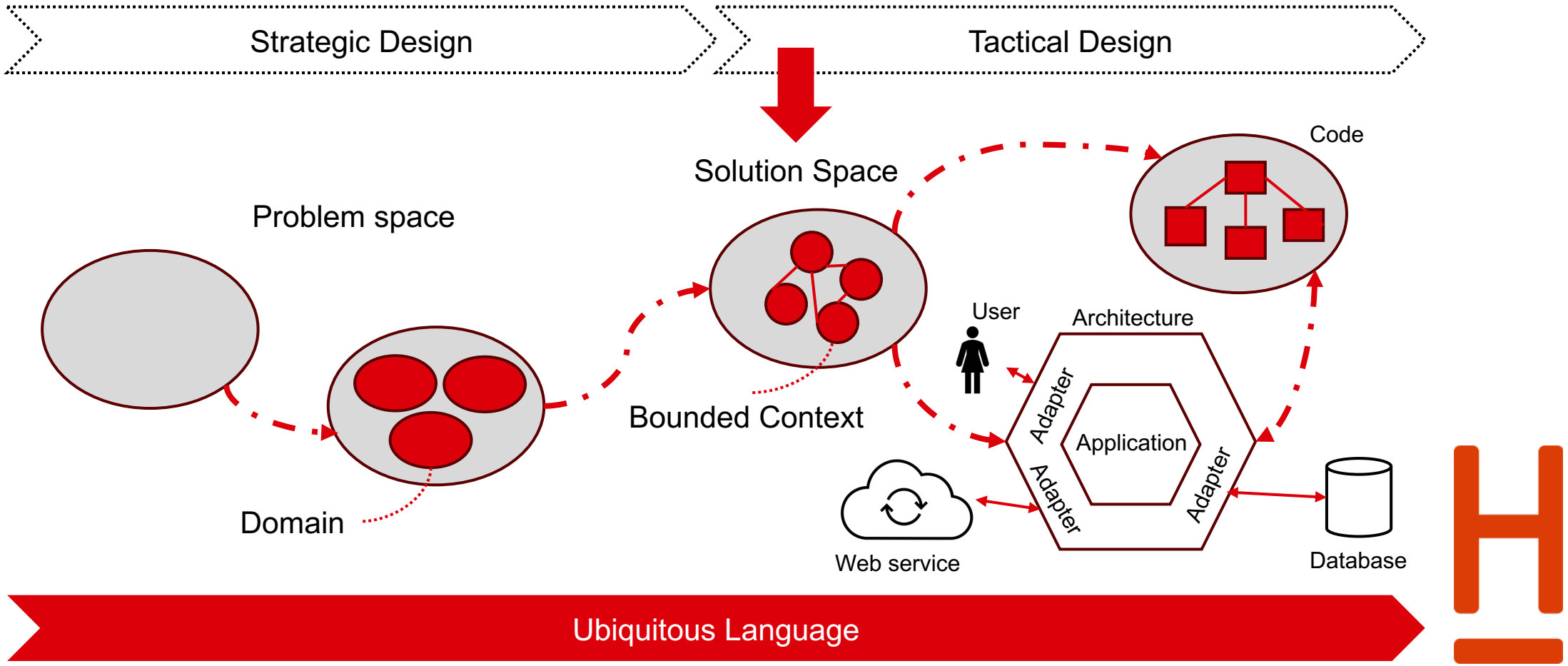
Domain Driven Design
- What is it?
- Effective Software Design
- Strategic Design
- Tactical Design

Strategic Design
- Subdomains
- Ubiquitous Language
- Bounded Context
- Context Mapping

Tactical Design
- Business Logic Pattern
- Architectural Pattern



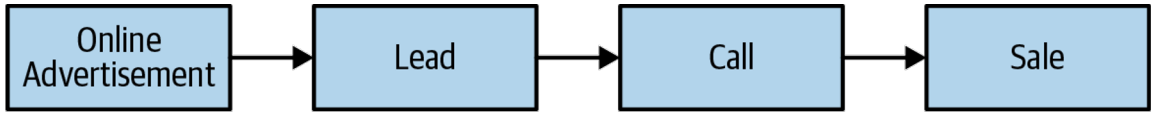
Domain Driven Design



Bounded Contexts

Mastering conceptual complexities

It is common for mental models of domain experts to be consistent only in a specific context.



Let's say we work for a telemarketing company. The company's marketing department generates leads through online ads. The sales department is responsible for getting potential customers to buy the products or services. Both departments have a different understanding of a lead.

Marketing Department

A lead represents an event that someone is interested in one of the products. The event of receiving the contact details of the potential customer is called a lead.

Sales Department

A lead represents the entire lifecycle of a sales process. It's not just an event, it's a long-lasting process.

Enterprise architecture management approaches often claim to develop a global "ubiquitous language" (without calling it that) for an entire company. However, these approaches often fail.



ShareWise Example



- The very same person using the app can be modelled differently in different context:
  - User (with an account, login)
  - Member (of a group, with invitation, a role in the group)
  - Participant (in one concrete expense)
  - Debtor, creditor (with a balance)

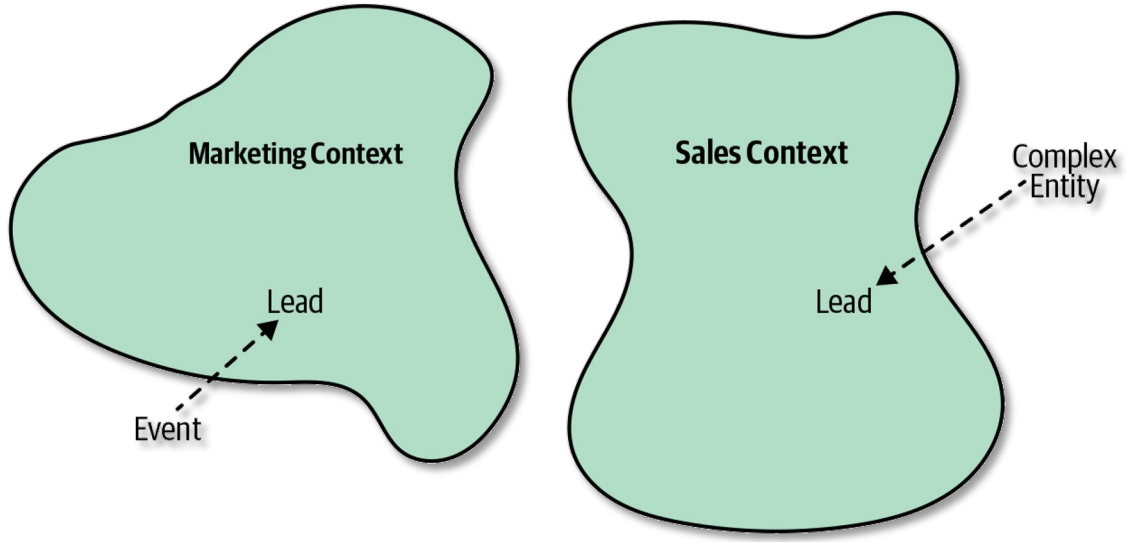


Bounded Context

What is a bounded context?

The solution to dealing with such problems is simple.

The goal is not to develop a single, all-encompassing ubiquitous language, but several smaller ones!



Bildquelle: Vladik Khononov, *What Is Domain-Driven Design?* O'Reilly Media, 2019

Bounded Context

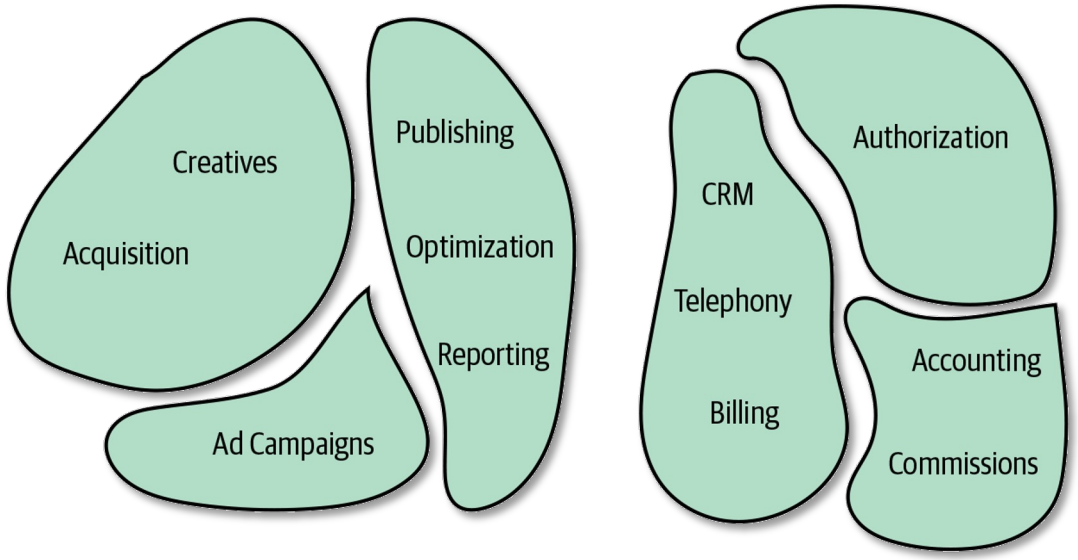
Scope of a Bounded Context

The size of a bounded context does not matter too much.

Models should never be large or small per se. Models must be clear and have a purpose.

The broader the boundary of a ubiquitous language, the more difficult it becomes to keep it consistent.

The decision on how large a bounded context is, depends on the specific problem domain.



- It can be beneficial for modeling to split a large ubiquitous language into smaller, more manageable problem domains.
- However, striving for small bounded contexts will also create more integration overhead, as we will see.

Bildquelle: Vladik Khononov, *What Is Domain-Driven Design?* O'Reilly Media, 2019

Bounded Context

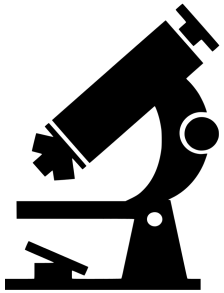
Bounded Contexts vs. Subdomains

Subdomains

The analysis phase involves identifying the different types of subdomains (Core, Supporting, Generic).

"Identification" means that the subdomains are already there; they are part of the business. Through analysis, we discover subdomains that already exist. We don't design them!

Subdomains help us to set priorities and use limited resources primarily in the core subdomains.





Bounded Contexts

Bounded contexts, on the other hand, are designed.

Choosing the boundaries of models is a strategic design decision. We decide how to divide the business domain into smaller, manageable problem domains.



ShareWise Bounded Contexts?



Context Mapping

Relations between bounded contexts

Models in different bounded contexts can be developed and implemented independently of each other.

But bounded contexts are not isolated, as a system cannot be built from independent components; the components must interact with each other in order to achieve the overarching system goals.

As a result, there will always be touchpoints, known as contracts.

Two bounded contexts use different ubiquitous languages.

Which language should be used for integration?

There are Domain-Driven Design's Patterns for handling relationships and integrations between bounded contexts and fall into categories:

- Cooperation
- Customer-Supplier
- Separate Ways

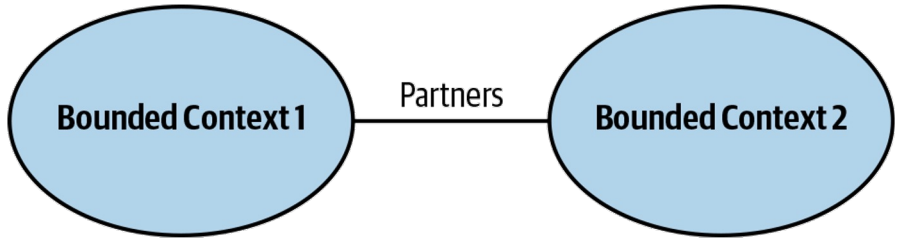


Context Mapping

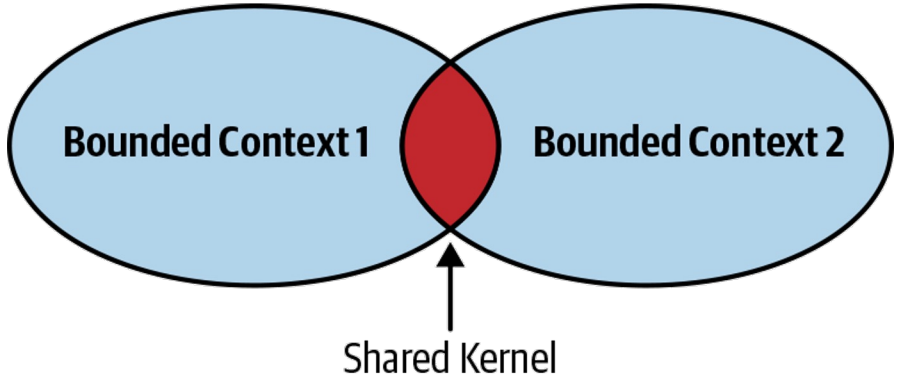
Cooperation Patterns

- Cooperation patterns refer to bounded contexts implemented by teams with well-established communication.
- E.g., bounded contexts implemented by the same team, teams with dependent goals, where the success of one team depends on that of the other and vice versa.

Partnership



Shared Kernel



Bildquelle: Vladik Khononov, *What Is Domain-Driven Design?* O'Reilly Media, 2019

Context Mapping

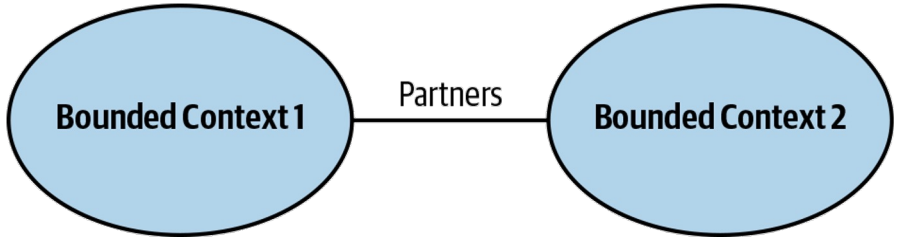
Cooperation Patterns – Partnership

In the partnership model, the integration between Bounded Context is coordinated on an ad hoc basis.

One team can notify a second team of a change in the API, and the second team will cooperate and make any necessary adjustments immediately (both ways).

No team dictates the language. The teams can choose the most appropriate solution integration cooperatively.

Neither team is interested in blocking the other.



Well-established collaboration practices, high levels of engagement, and frequent synchronization between teams are required for this integration pattern.
This pattern is usually not suitable for geographically dispersed teams, as it can lead to synchronization and communication issues.

Bildquelle: Vladik Khononov, *What Is Domain-Driven Design?* O'Reilly Media, 2019

Context Mapping

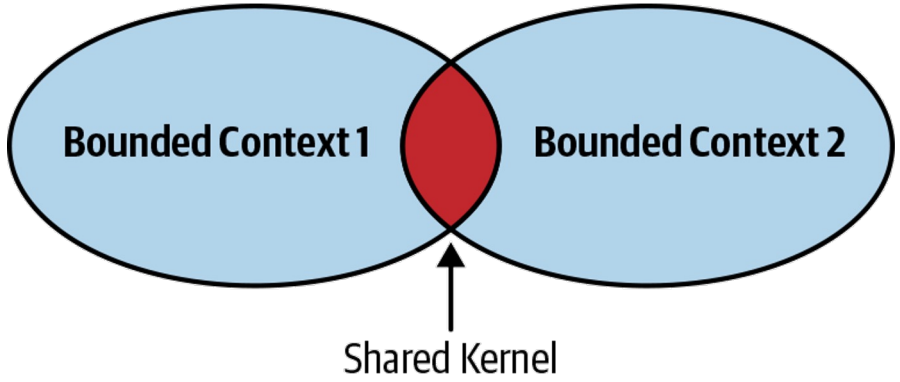
Cooperation Patterns – Shared Kernel

The shared kernel is a more formal way of defining a contract between bounded contexts without a power imbalance.

The contract is explicitly defined in a compiled library - the common kernel.

Shared kernels contradict a core principle of bound contexts: Only one team should own a bounded context.

The key to implementing the shared kernel pattern is to keep the scope of the shared kernel small and limit it to the integration contract only.



Changing the contract can interrupt other team's build: this pattern requires a high level of synchronization between teams.

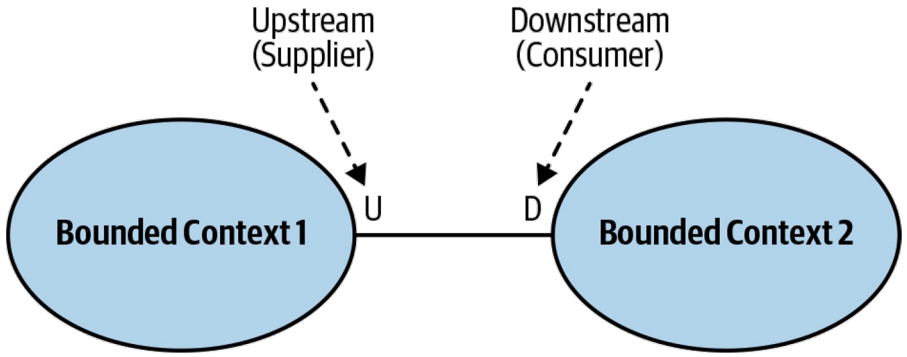
Shared kernels can be a disciplining prerequisite for the integration of bounded contexts that owned by the same team.

Bildquelle: Vladik Khononov, *What Is Domain-Driven Design?* O'Reilly Media, 2019

Context Mapping

Customer-Supplier Patterns

- Unlike in the case of cooperation, both teams (upstream and downstream) can succeed independently of each other.
- Therefore, in many cases, there is an imbalance of power: either the upstream or downstream team can dictate the integration contract.
- DDD envisages the three patterns to address such power inequalities.



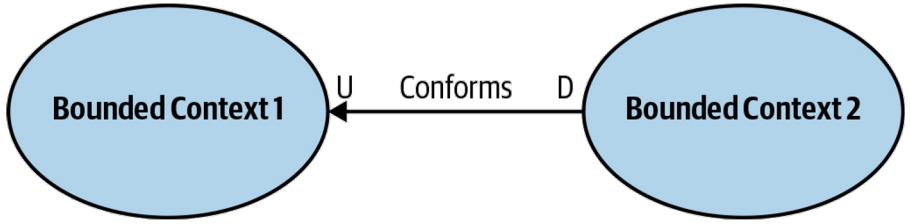
1. Conformist Pattern
2. Anti-Corruption Layer
3. Open-Host Service

Bildquelle: Vladik Khononov, *What Is Domain-Driven Design?* O'Reilly Media, 2019

Context Mapping

Customer-Supplier Patterns – Conformist Pattern

- The balance of power is in favor of the upstream team, which is not motivated to support the needs of its customers.
- It only provides the integration contract, which is defined according to its own model - take it or leave it.
- Such power imbalances often exist among external service providers but can also be caused by internal organizational policies.
- If the downstream team can accept the upstream team's model, the relationship between the bounded contexts is said to be conformist.



The Downstream Team conforms to the model of the Upstream Team.

The downstream team's decision to give up some of its autonomy can make perfect sense. For example, the contract disclosed by the upstream team may be an industry-standard, well-established model.

Bildquelle: Vladik Khononov, *What Is Domain-Driven Design?* O'Reilly Media, 2019

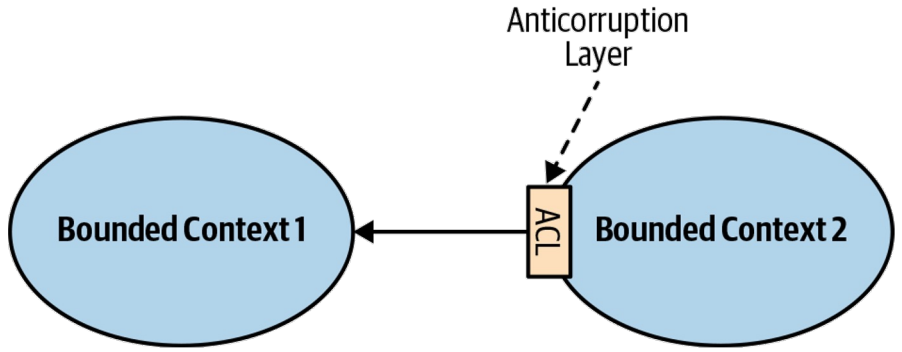
Context Mapping

Customer-Supplier Patterns – Anti Corruption Layer

The Consumer is not willing to accept the supplier's model unconditionally.

The balance of power is still shifted in favor of the upstream service. However, the downstream bounded context is not ready to adapt.

Instead, the model of the upstream bounded context is translated via an anti-corruption layer into a model that is tailored to the needs of one's own bounded context.



The Anticorruption Layer Pattern addresses scenarios where it is not desirable or worth the effort to fully conform to the supplier's model

- The downstream bounded context contains a core subdomain - adherence to a supplier's model could hinder the modeling of one's own problem domain.
- If the upstream model is a legacy model (e.g., poor or unsuitable).
- If the supplier's contract changes frequently
- If the supplier's model is very large and only parts of this model are needed.

Bildquelle: Vladik Khononov, *What Is Domain-Driven Design?* O'Reilly Media, 2019

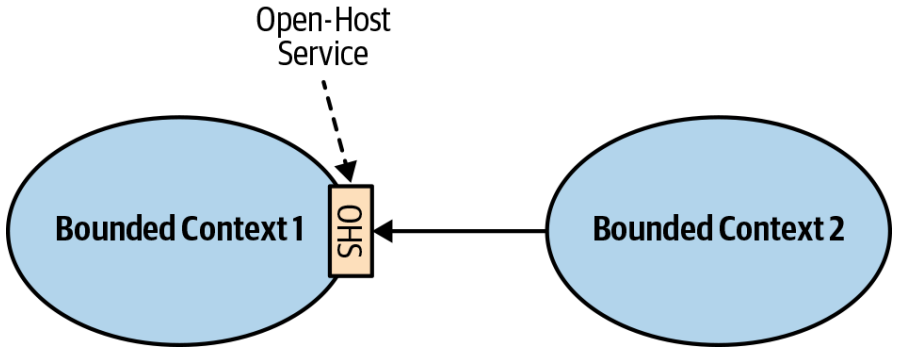
Context Mapping

Customer-Supplier Patterns – Open Host Service

Power is shifted towards consuming services.

The supplier is interested in protecting consumers (from changes) and providing the best possible service.

The upstream supplier decouples its implementation from the public interface. This decoupling allows the supplier to evolve its implementation model and public model at different speeds



The provider's public interface is not intended to correspond to its own ubiquitous language but is intended to provide consumers with a convenient protocol expressed in an integration-oriented language. Therefore, the public protocol is referred to as a "published language".

Bildquelle: Vladik Khononov, *What Is Domain-Driven Design?* O'Reilly Media, 2019

Context Mapping

Separate Ways Pattern

The last option for integration is not to collaborate at all.

Teams are not willing or able to work together.

Should never be use on bounded contexts belonging to a core subdomain

(1) Problems with communication
A common reason for avoiding collaboration is communication difficulties caused by the size of the organization or internal political issues. In these cases, it may be more cost-effective to duplicate functionality in multiple bounded contexts.

(2) Generic Subdomains (Libraries)
If the subdomain in question is generic, if the generic solution is easy to integrate, it may be more cost-effective to integrate it locally in each of the bounded contexts. One example is a logging framework; it would make little sense to provide it as a service. Duplicating functionality across multiple bounded would be an efficient solution.

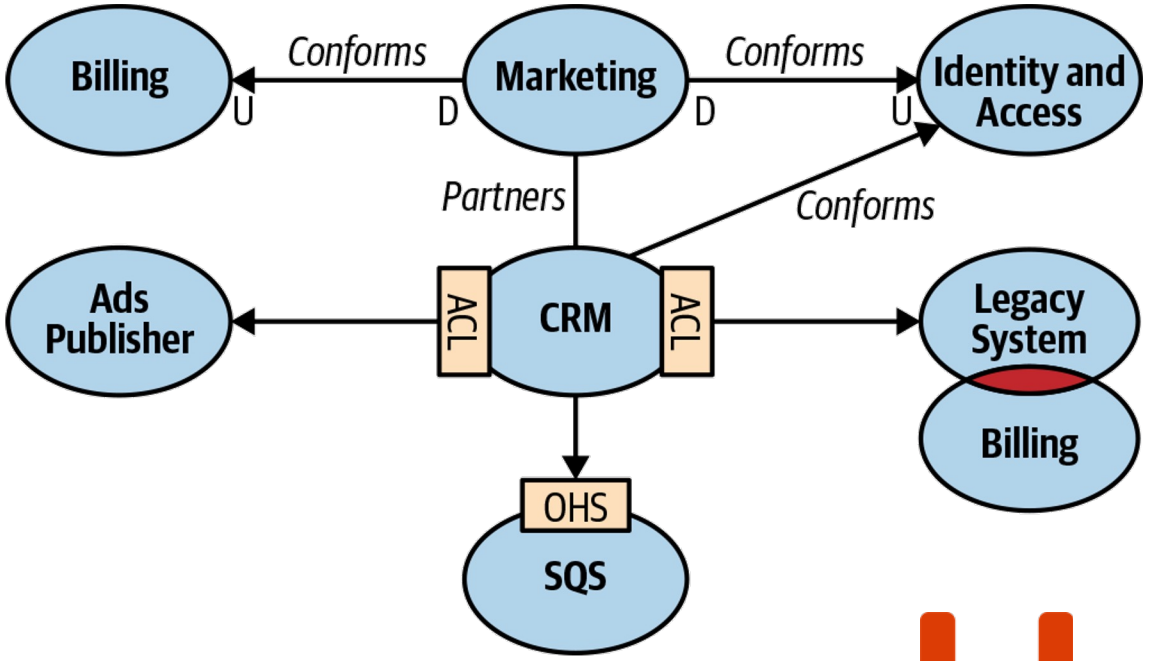
(3) Model differences
When models of bounded contexts are so different that a compliant relationship is not possible, and implementing an anti-corruption layer would be more expensive than duplicating functionality.

Context Maps

Context maps are a top-level visual representation of all bounded contexts of a system.

This simple visual notation already provides valuable strategic insights on several levels:

- **High-level design:** A context map provides an overview of the system's components and models.
- **Communication patterns** between the teams are shown and which teams work together.
- **Organizational issues:** Context maps provide insight into organizational issues. What does it mean when downstream consumers of a particular upstream team all rely on anti-corruption layers, or when all separate ways are focused on the same team?



Bildquelle: Vladik Khononov, *What Is Domain-Driven Design?* O'Reilly Media, 2019

ShareWise Context Mapping?

Part of the exercises.

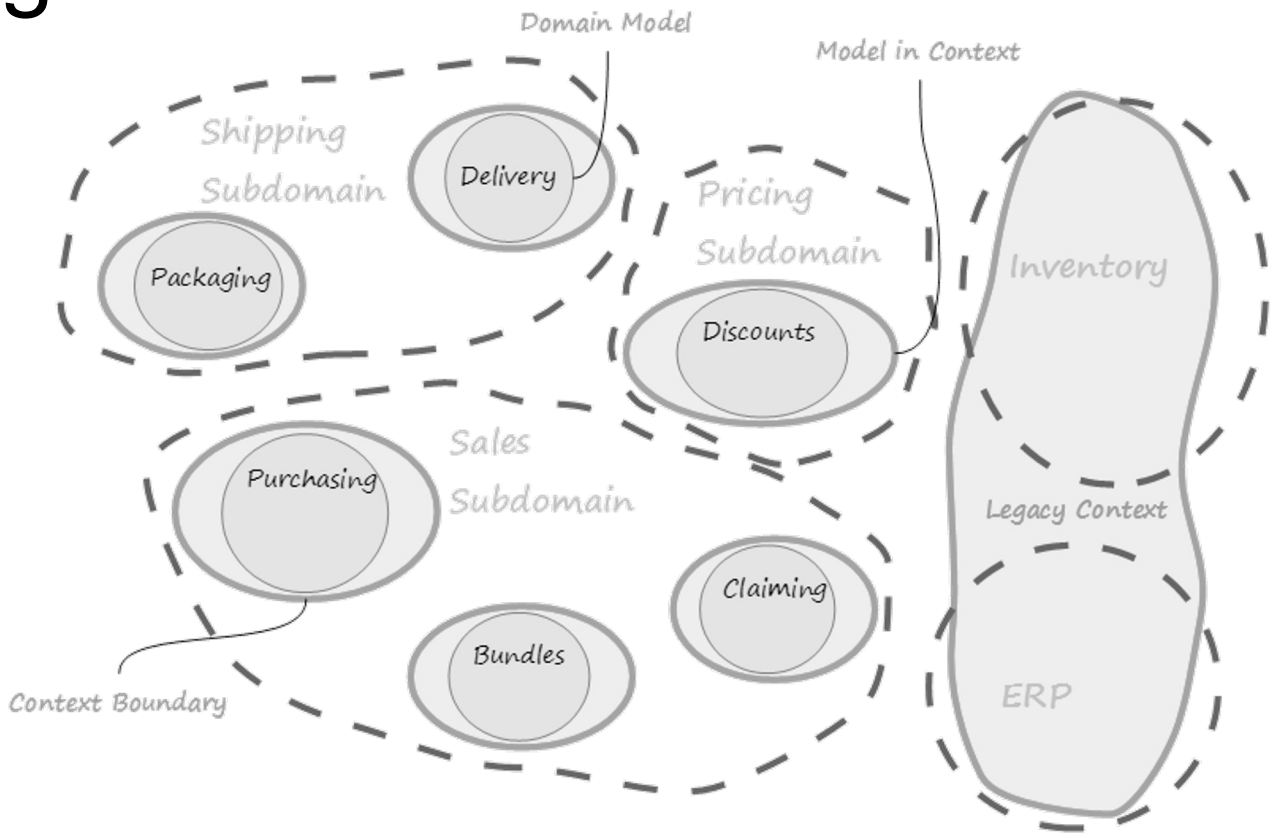


Domain Driven Design

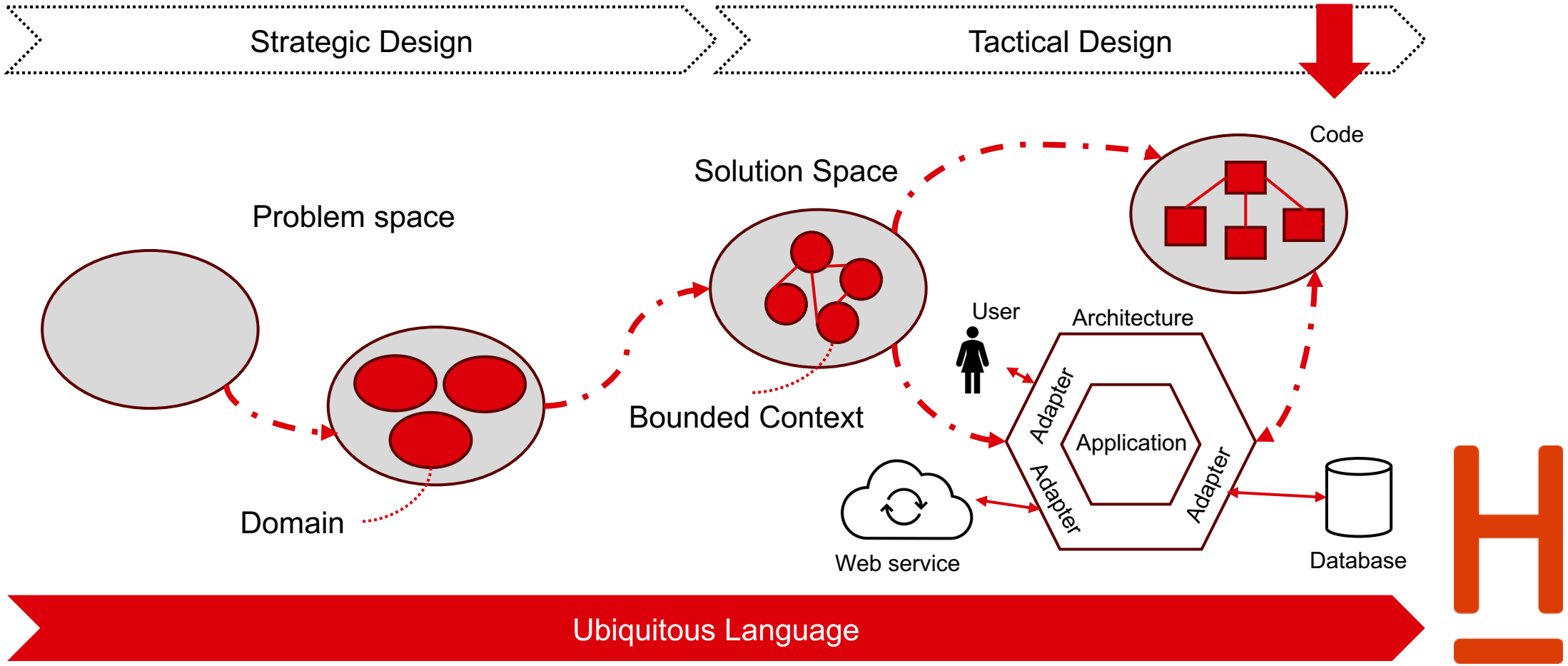
Domain Driven Design
- What is it?
- Effective Software Design
- Strategic Design
- Tactical Design

Strategic Design
- Subdomains
- Ubiquitous Language
- Bounded Context
- Context Mapping

Tactical Design
- Business Logic Pattern
- Architectural Pattern

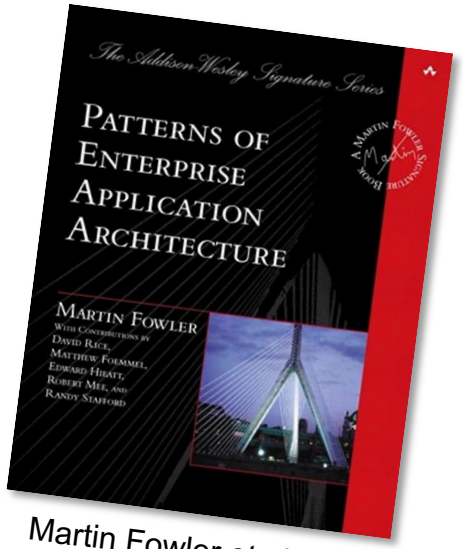


Domain Driven Design



Tactical Design

- So far, we have dealt with strategic design decisions
- Tactical design is about the implementation of components (bounded contexts) – not the entire system architecture
- Design patterns for the tactical design are discussed
- The patterns are presented on an overview level to explain differences and suitable areas of application



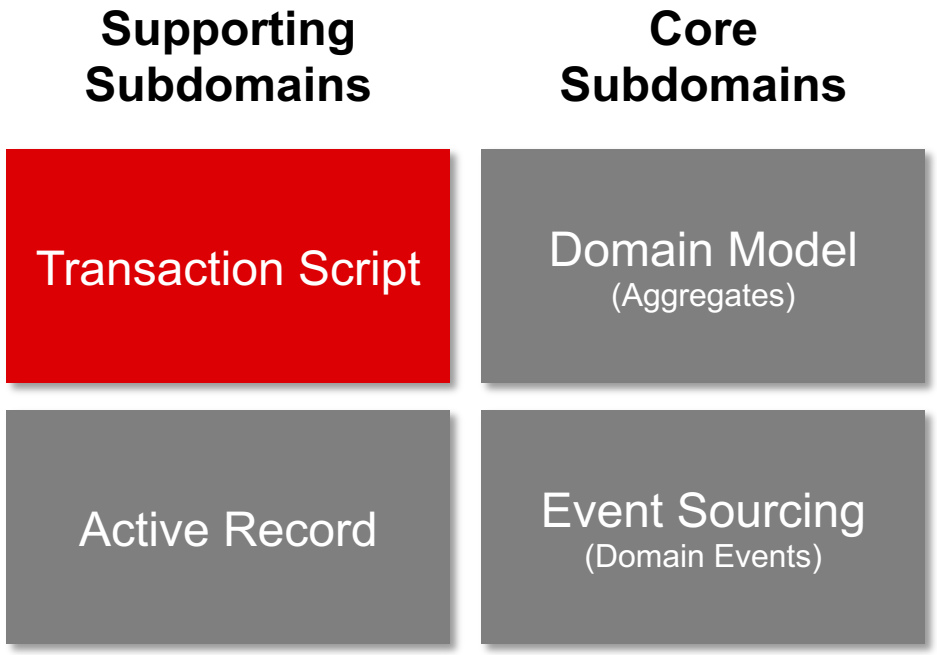
Martin Fowler et al., 2002



Tactical Design

Business Logic Implementation Patterns

- Not all subdomains are created equal
- Different subdomains have different strategic importance and complexity
- Each pattern is suitable for a different level of complexity in the business domain



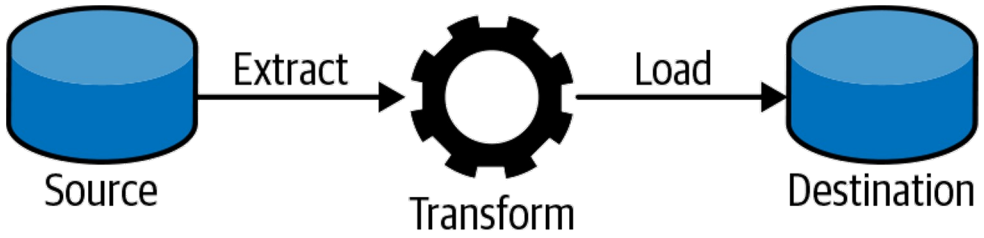


Business Logic Pattern

Supporting Subdomains: Transaction Script

Extract – Transform - Load

This pattern works well for the simplest problem domains, where the business logic is similar to extract-transform-load (ETL) operations, which means that each operation extracts data from a source, applies transformation logic to convert it to another form, and loads the result into the target store.

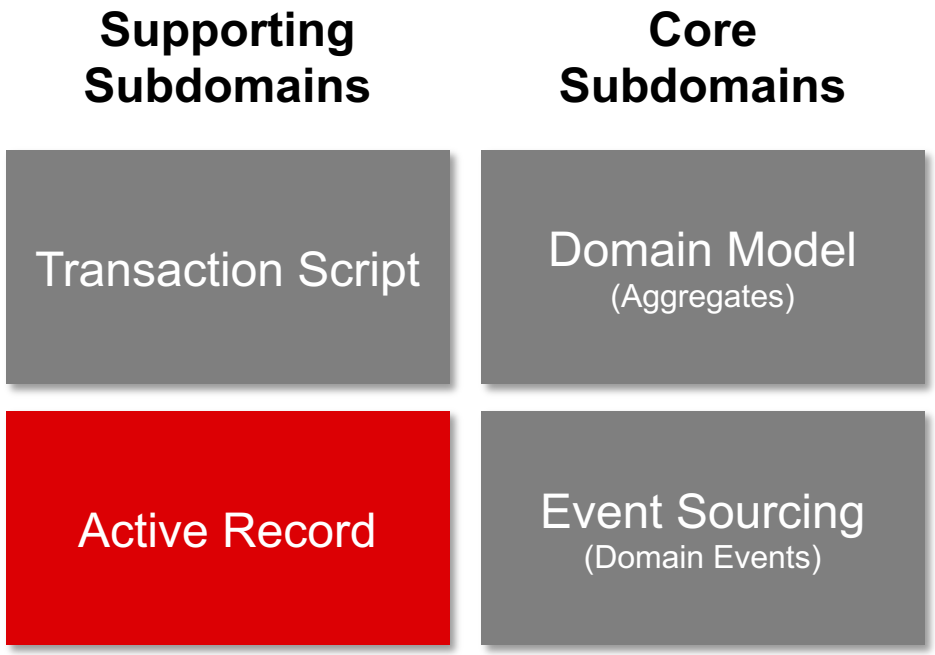


- Business logic is based on simple procedures
- Each procedure is implemented as a script
- Thin abstraction layer for integration with memory mechanisms is optional
- Direct access to databases is allowed
- The only requirement is transactional behavior: operations must succeed or fail
- If the system fails, it remains in a consistent state
- Hence the name of the pattern: Transaction Script

Bildquelle: Vladik Khononov, *What Is Domain-Driven Design?* O'Reilly Media, 2019

Tactical Design

Business Logic Implementation Patterns



Business Logic Pattern

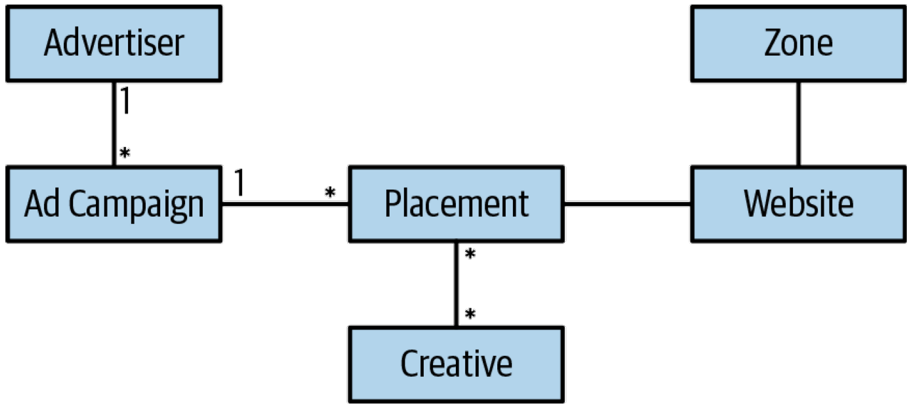
Supporting Subdomains: Active Record

Like the previous pattern, this pattern also supports cases where the business logic is simple. However, here, the business logic can operate on more complex data structures. For example, instead of flat datasets, we can have more complicated object trees and hierarchies.

```
public class CreateUser {
    public void Execute(userDetails) {
        try {
            DB.StartTransaction();

            var user = new User();
            user.Name = userDetails.Name;
            user.Email = userDetails.Email;
            user.Save();

            DB.Commit();
        } catch {
            DB.Rollback();
        }
    }
}
```

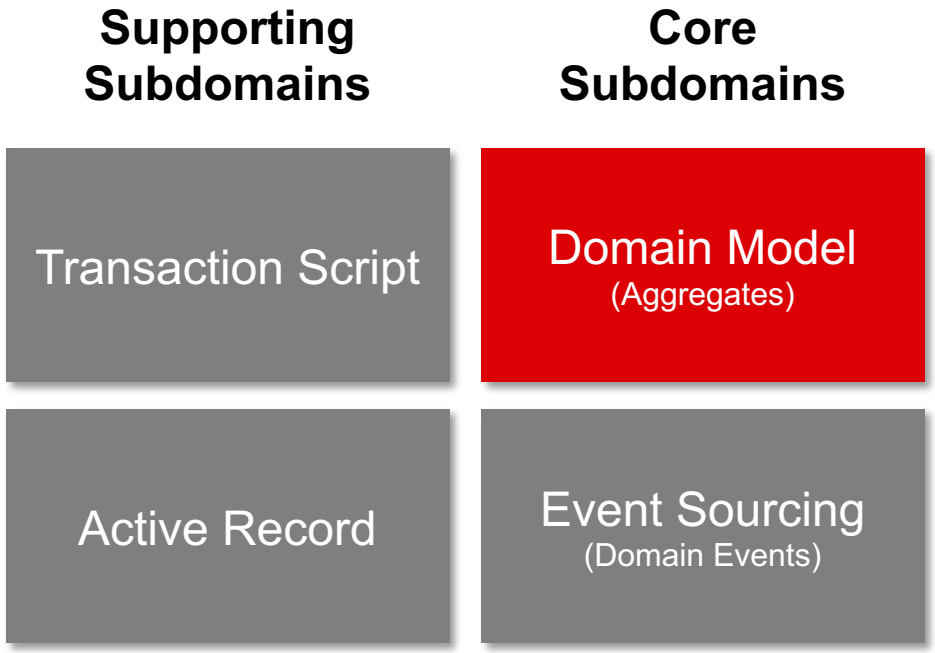


- For more complex data structures, transactional scripts result in a lot of repetitive code
- Active records represent data structure and implement required CRUD operations
- However, objects depend on Object Relational Mapping Frameworks (ORM) or other data access frameworks
- Access is not directly to the database, but via Active Records.
- The name of the pattern is derived from the fact that each record is "active" and implements the required data access logic.

Bildquelle: Vladik Khononov, *What Is Domain-Driven Design?* O'Reilly Media, 2019

Tactical Design

Business Logic Implementation Patterns



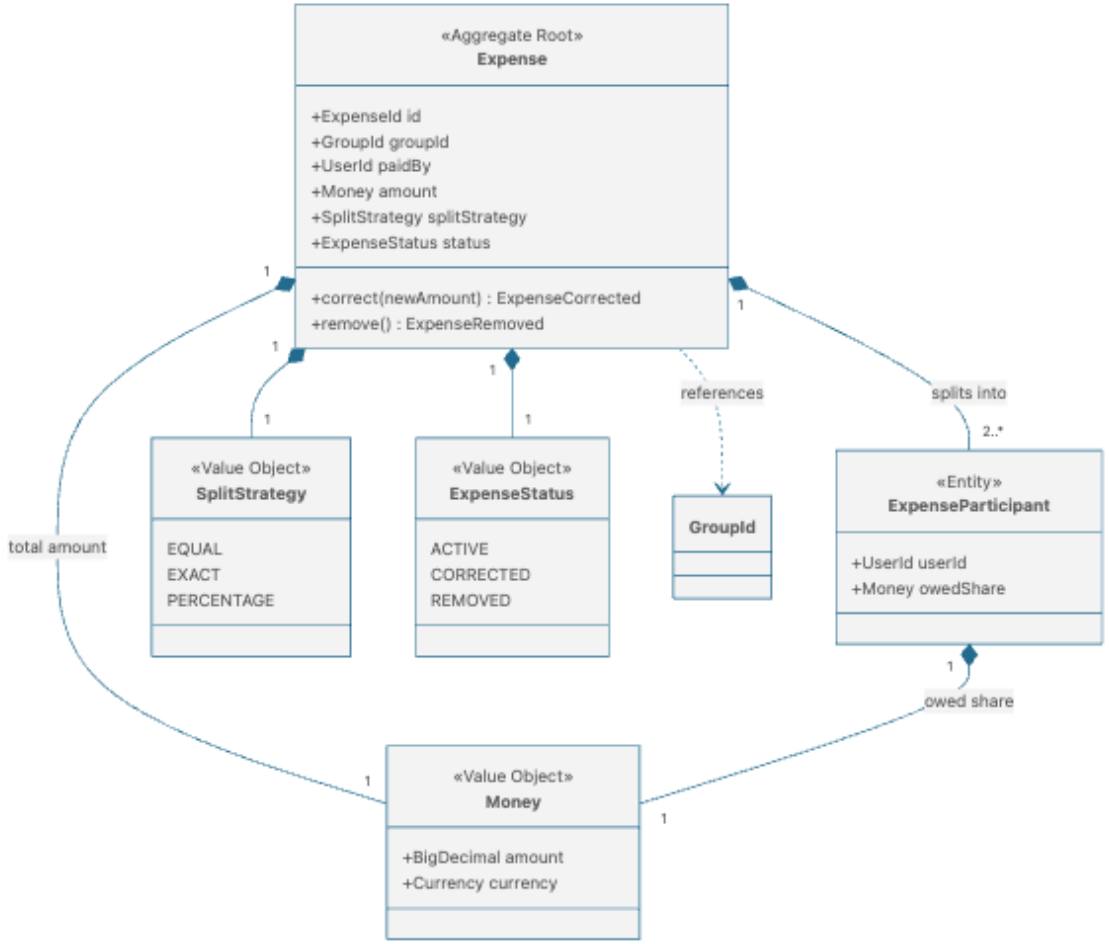
Business Logic Pattern

Core Subdomains: Domain Model

According to Martin Fowler's definition, a domain model is an object model of a domain that contains both behavior and state (data). The tactical patterns of DDD provide the following building blocks, among others:

- Value Object
- Aggregate

The domain model pattern allows the code to "speak" the Ubiquitous Language and conceptually follow the mental models of the domain experts.



Value Object

- A Value Object, or simply a Value, models an immutable conceptual whole.
- It does not have a unique identity
- Equality is determined by comparing the attributes encapsulated by the Value type.
- It is not a thing but is often used to describe, quantify, or measure an Entity

```
public class Color
{
    public readonly byte Red;
    public readonly byte Green;
    public readonly byte Blue;

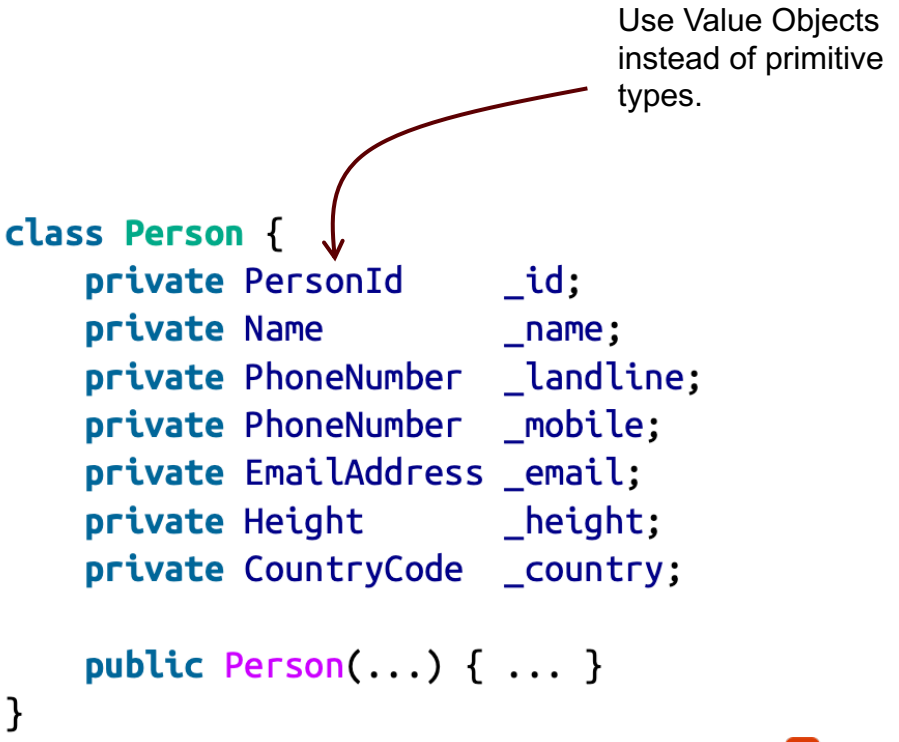
    public Color(byte r, byte g, byte b)
    {
        this.Red = r;
        this.Green = g;
        this.Blue = b;
    }

    public Color MixWith(Color other)
    {
        return new Color(
            r: (byte) Math.Min(this.Red + other.Red, 255),
            g: (byte) Math.Min(this.Green + other.Green, 255),
            b: (byte) Math.Min(this.Blue + other.Blue, 255)
        );
    }

    ...
}
```

Entities

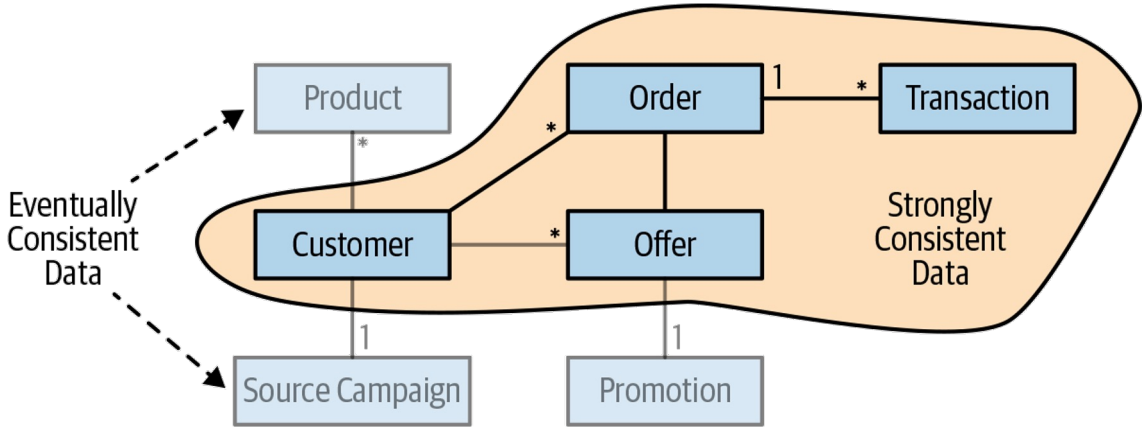
- An Entity models an individual thing.
- Each Entity has a unique identity in that you can distinguish its individuality from among all other Entities of the same or a different type.
- Many times, an Entity will be mutable: its state will change over time.
- Implemented in the context of an Aggregate





Aggregate

- Each Aggregate is composed of one or more Entities, where one Entity is called the Aggregate Root.
- Aggregates may also have Value Objects composed on them.
- The Root Entity of each Aggregate owns all the other elements clustered inside it.
- The name of the Root Entity is the Aggregate's conceptual name.
- Each Aggregate forms a transactional consistency boundary.



Rule 1:
Protect functional invariants within aggregate boundaries by means of interfaces

Rule 2:
Design small aggregates

Rule 3:
Reference others aggregates only by their identity

Rule 4:
Update other aggregates using Eventual Consistency

Bildquelle: Vladik Khononov, *What Is Domain-Driven Design?* O'Reilly Media, 2019

ShareWise



- Entity
  - Group
  - Expense
  - SettlementPayment
  - …
- Value Object
  - Money
  - ExpenseShare
  - Percentage
  - GroupName

ShareWise



Expense Aggregate

- ExpenseId
- GroupId
- PayerId
- Money
- ExpenseShares
- ExpenseStatus

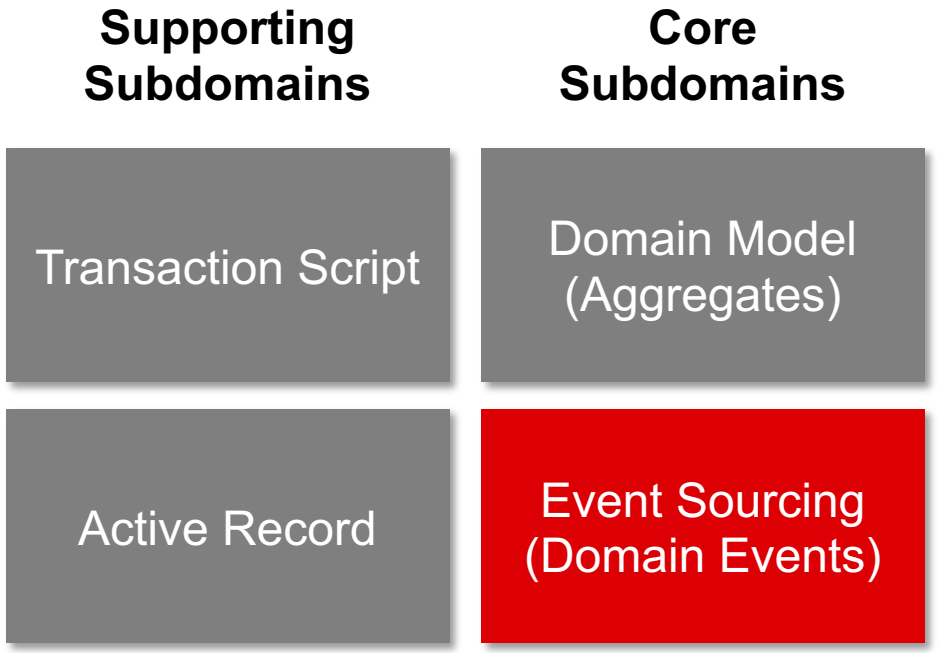
Possible invariants

- The amount is positive.
- The sum of the shares corresponds to the total amount.
- The payer belongs to the group.
- All those involved belong to the group.
- A finalized issue may not be changed directly.



Tactical Design

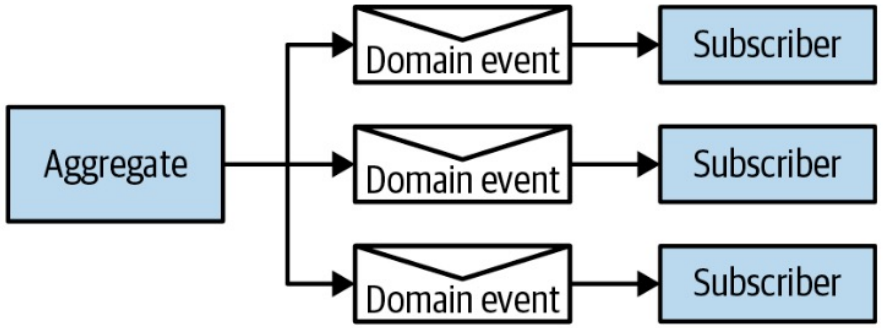
Business Logic Implementation Patterns

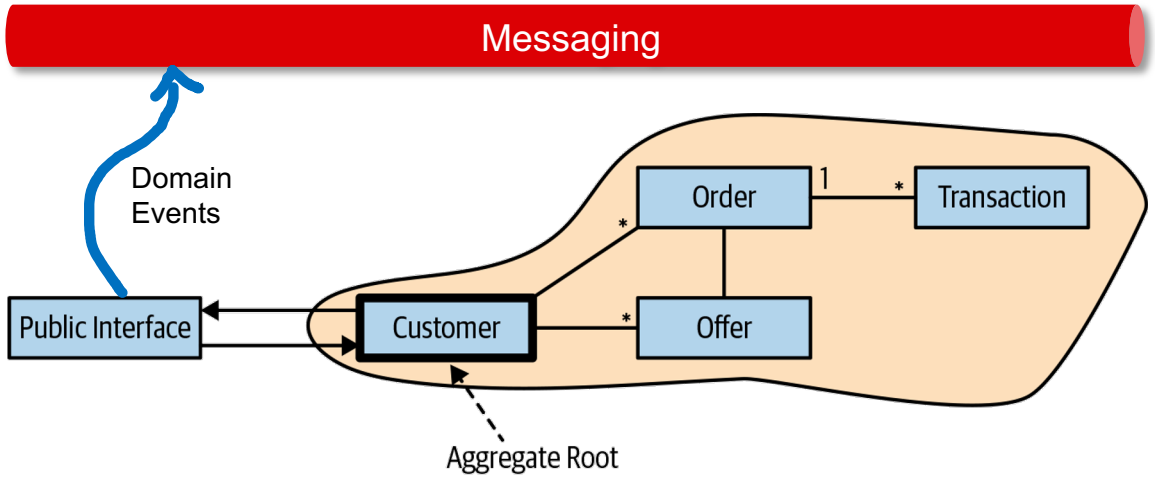


Business Logic Pattern

Core Subdomains: Domain Events

- A domain event is a message that describes a significant event that occurred in the business domain.
- For example: order paid, stock replenished, advertising campaign published, etc.
- Such events are part of an aggregate's public interface. An aggregate publishes its domain events through its public interface.



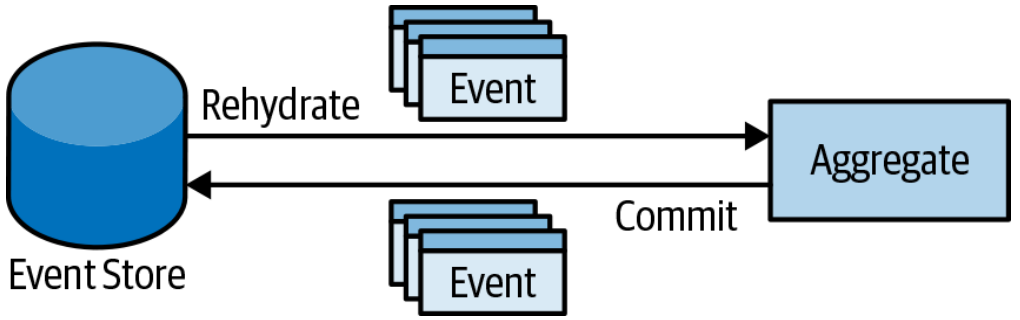


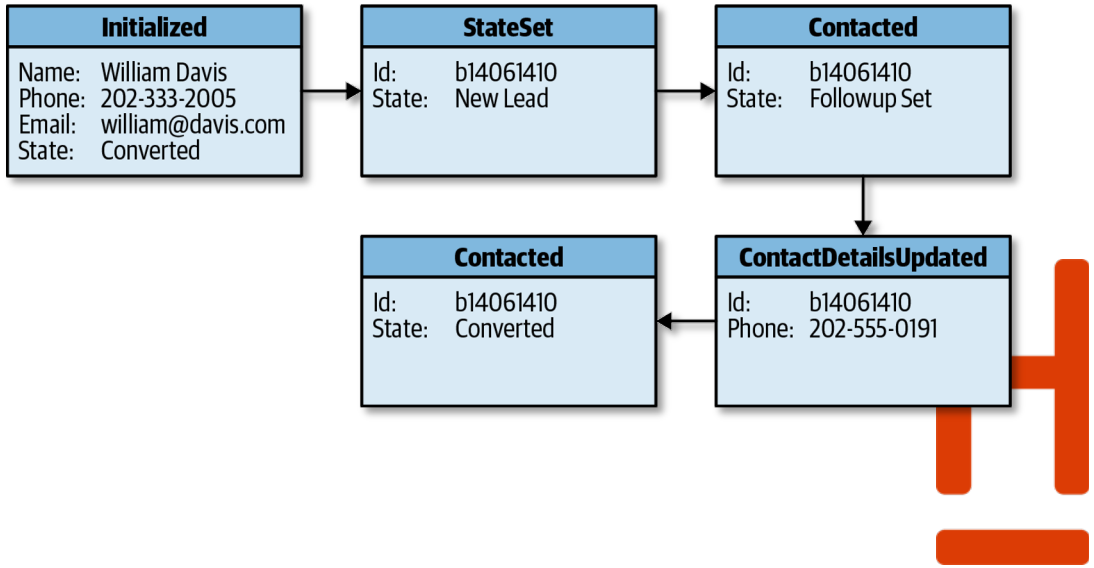
Bildquelle: Vladik Khononov, *What Is Domain-Driven Design?* O'Reilly Media, 2019

Business Logic Pattern

Core Subdomains: Event Sourcing

- The event sourcing pattern uses domain events to introduce the dimension of time into the domain model.
- Any change in the state of the system should be expressed and recorded as a domain event.
- The database that stores the system's domain events is the only highly consistent store - the system's "single source of truth".
- Stored events are used to recreate the state of an aggregate (rehydrate)
- Event sourcing is used, for example, in the financial industry to represent changes in a ledger. A ledger is an append-only log of transactions. A current state (e.g. the account balance) can always be derived by replaying the ledger records. (Audit-safe)





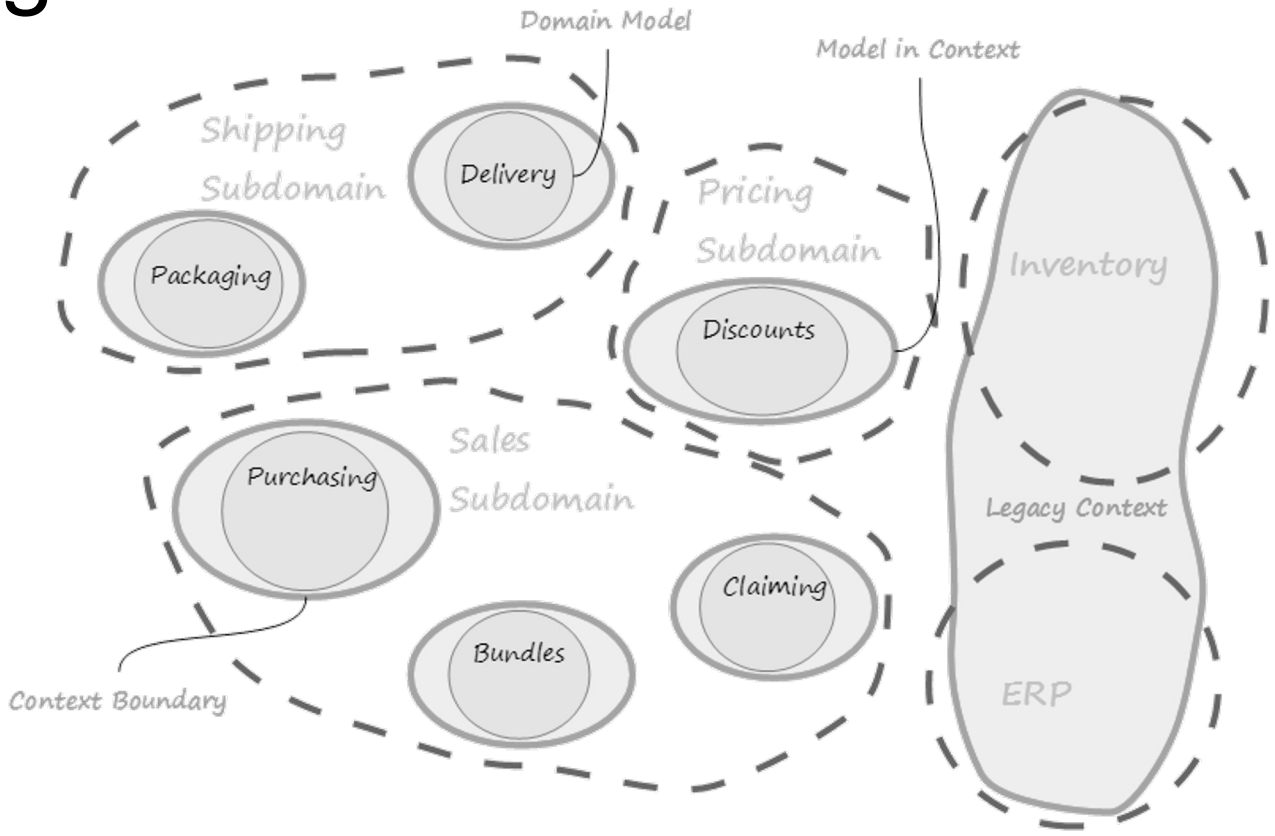
Bildquelle: Vladik Khononov, *What Is Domain-Driven Design?* O'Reilly Media, 2019

Domain Driven Design

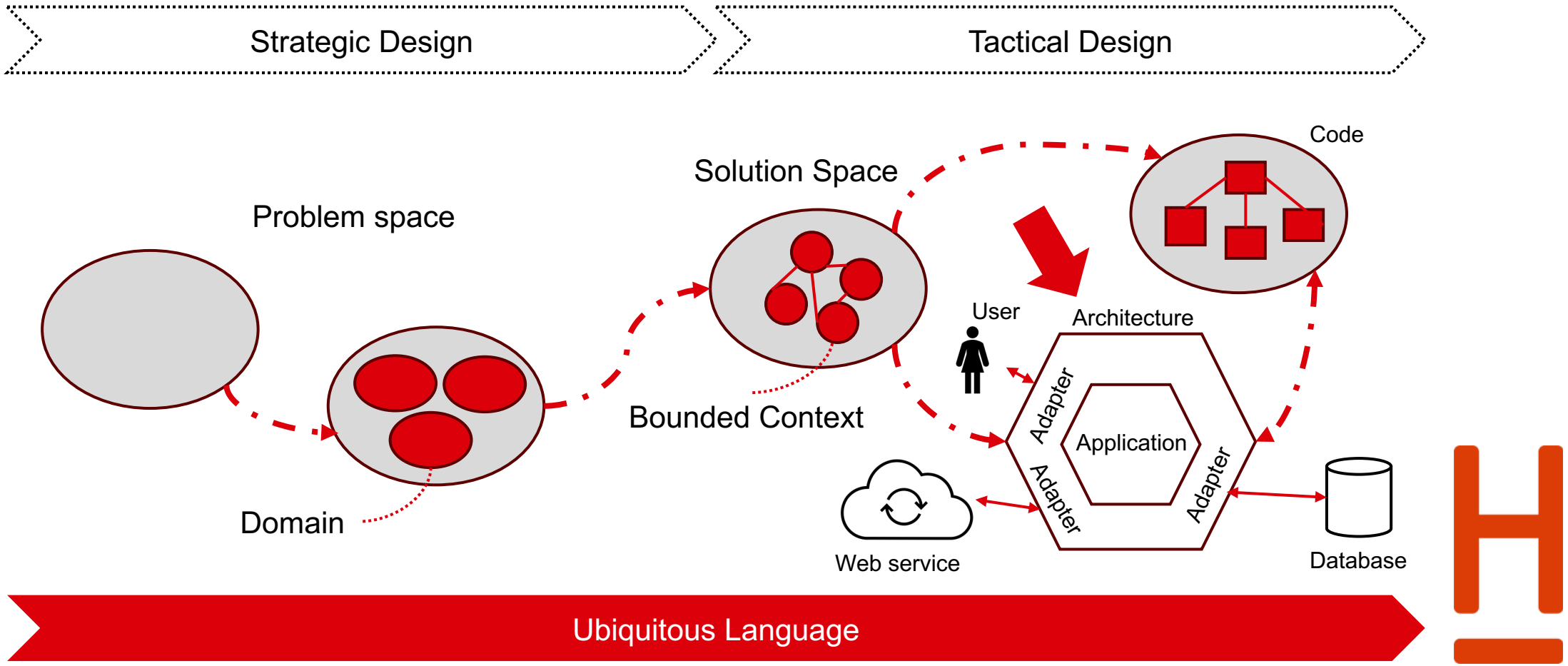
Domain Driven Design
- What is it?
- Effective Software Design
- Strategic Design
- Tactical Design

Strategic Design
- Subdomains
- Ubiquitous Language
- Bounded Context
- Context Mapping

Tactical Design
- Business Logic Pattern
- Architectural Pattern



Domain Driven Design



Tactical Design

Architectural Patterns

- Which architectural pattern to use is an important tactical design decision.
- The right pattern supports the implementation of the functional and non-functional requirements of the system.
- We will look at three common architectural patterns and suitable use cases for them.

1. Layered Architecture
2. Ports & Adapters
3. CQRS (Command-Query-Responsibility-Segregation)



Tactical Design

Layered Architecture

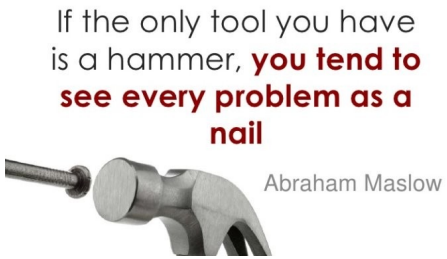


Presents and defines the user interface

Implements business logic

Access to persistence mechanisms (DBs and other infrastructure components)

A classic! Maybe even THE classic!



The dependency between the business logic and the data access layers makes this architectural pattern a good fit for a system whose business logic has been implemented with the Active Record pattern.

The pattern is therefore often found in bounded contexts for supporting domains!

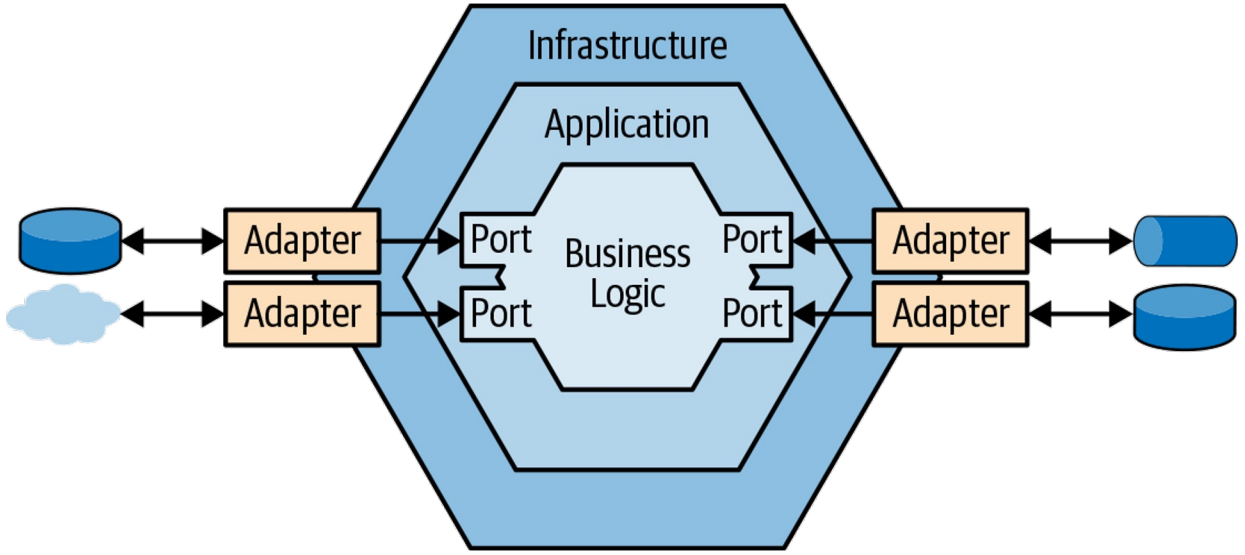


Bildquelle: Vladik Khononov, *What Is Domain-Driven Design?* O'Reilly Media, 2019

Tactical Design

Ports & Adapters (also Hexagonal Architecture)

- The Ports & Adapter pattern aims to decompose the code base according to technological aspects
- Terminology: "Infrastructure" refers to both user interface, data access, and other infrastructural concerns
- Dependencies: Business logic at the center, this should not be directly dependent on infrastructure (dependency inversion)
- Application layer: Public interface facade, orchestrates business logic
- Decoupling business logic from all technological concerns



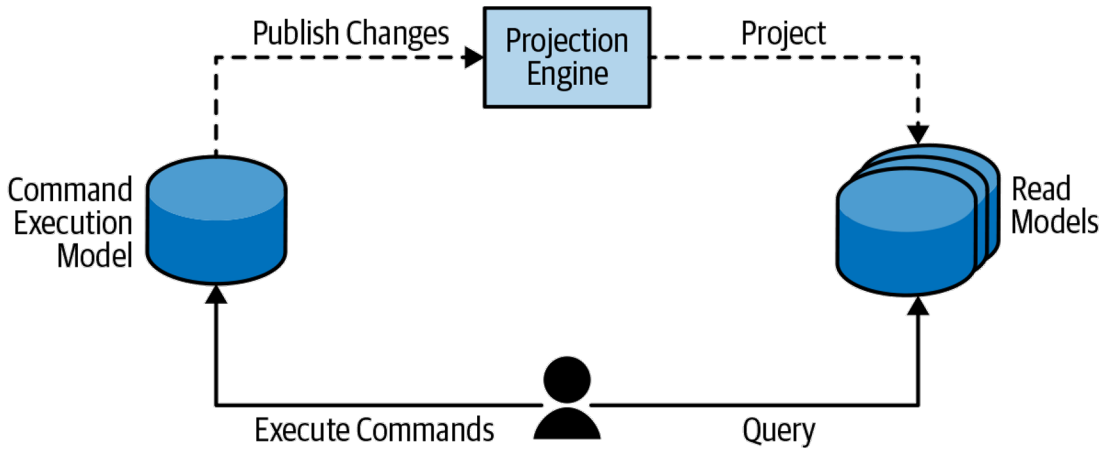
The pattern is therefore often found in bounded contexts for core domains!

Bildquelle: Vladik Khononov, *What Is Domain-Driven Design?* O'Reilly Media, 2019
See als: Alistair Cockburn - https://alistair.cockburn.us/hexagonal-architecture/

Tactical Design

CQRS (Command-Query Responsibility Segregation)

- CQRS is based on a single model for performing operations that change the state of the system (write).
- Command Execution Model is the only model that represents highly consistent data (single source of truth).
- The system can generate any number of models using projections that are necessary to represent data to users or other systems (read). These models are read-only. None of the system's operations can directly change the data of the read models.
- Supports Polyglot modeling well
- Using event sourcing, an aggregate's state needs to be hydrated and cannot be flexibly queried, CQRS provides a solution with read models



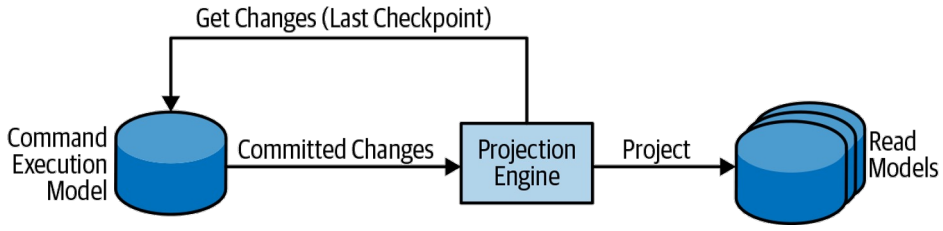
CQRS is often found in bounded contexts with event sourcing of core subdomains!

See also: Greg Young, CQRS Documents - https://cqrs.files.wordpress.com/2010/11/cqrs_documents.pdf

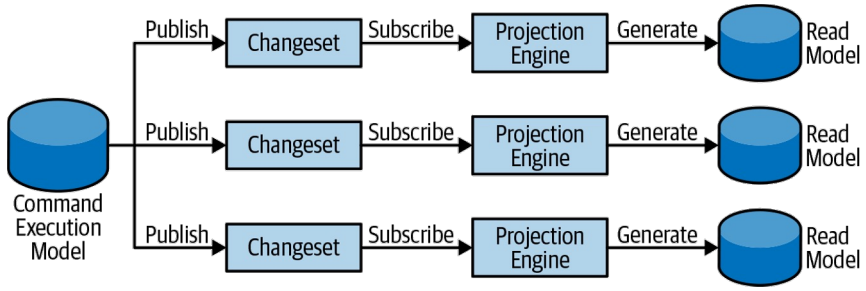
Tactical Design

CQRS (Projections)

SYNCHRONOUS PROJECTIONS



ASYNCHRONOUS PROJECTIONS



Tip:
It is advisable to use a synchronous projection as a basis and only add an additional asynchronous projection if necessary.
Why?

Bildquelle: Vladik Khononov, *What Is Domain-Driven Design?* O'Reilly Media, 2019

Domain Driven Design

